

Q1. 請參照附圖的說明，撰寫 Python 程式以呈現下列各式卷積的影像處理結果。 1-1、將影像灰階化；1-2、模糊化；1-3、銳化；1-4、Laplacian 邊緣偵測；1-5、Sobel X 軸邊緣偵測；1-6、Sobel Y 軸邊緣偵測。

程式碼如下:

```
import numpy as np
import cv2
from skimage import data
from skimage.exposure import rescale_intensity

def convolve(image, kernel):
    (iH, iW) = image.shape[:2]
    (kH, kW) = kernel.shape[:2]
    pad = (kW - 1) // 2 # 卷積核的一半大小
    image = cv2.copyMakeBorder( # 以複製邊界方式補邊
        image, pad, pad, pad, pad, cv2.BORDER_REPLICATE
    )
    output = np.zeros((iH, iW), dtype="float32")

    for y in np.arange(pad, iH + pad):
        for x in np.arange(pad, iW + pad):
            roi = image[y - pad:y + pad + 1, x - pad:x + pad + 1]
            k = roi * kernel # 對應元素相乘
            output[y - pad, x - pad] = k.sum() # 再把結果加總

    # 將結果縮放回 0~255 並轉成 8-bit 影像
    output = rescale_intensity(output, in_range=(0, 255))
    output = (output).astype("uint8")
    return output
```

```
pip install opencv-python scikit-image
```

```
... Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.12/dist-packages (0.25.2)
Requirement already satisfied: numpy<2.3.0,>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
Requirement already satisfied: scipy>=1.11.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (1.16.3)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (2.37.2)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (2025.10.16)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (0.4)
```

```
pip install opencv-python
```

```
... Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: numpy<2.3.0,>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
```

1-1、將影像灰階化

```
import skimage
import cv2
from google.colab.patches import cv2_imshow # 專門給 Colab
用的顯示函式
image = skimage.data.chelsea() # 讀取範例
彩色影像 (RGB)
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY) # 轉成灰
階
cv2_imshow(image)
cv2_imshow(gray)
```



1-2、模糊化

```
import skimage
import cv2
from google.colab.patches import cv2_imshow # Colab 專用顯示函式
# 讀取 skimage 範例影像 (RGB)
image = skimage.data.chelsea()
# 小遮罩：7x7 模糊
small_blur = cv2.blur(image, (7, 7))
# 大遮罩：21x21 模糊
large_blur = cv2.blur(image, (21, 21))
print("小模糊:7x7")
cv2_imshow(small_blur)
print("大模糊:21x21")
cv2_imshow(large_blur)
```

小模糊:7x7



大模糊:21x21



1-3、銳化

```
image = skimage.data.chelsea()  
# 將 RGB 彩色影像轉成灰階影像，方便之後只針對亮度做處理  
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)  
# 定義一個 3x3 的銳化卷積核（中心加強、周圍減弱，用來讓邊緣變清晰）  
kernel = np.array([[0, -1, 0],  
                    [-1, 5, -1],  
                    [0, -1, 0]], dtype=np.float32)  
sharp_gray = cv2.filter2D(gray, ddepth=-1, kernel=kernel)  
  
print("Sharpen Gray")  
cv2_imshow(sharp_gray)
```

Sharpen Gray



1-4、Laplacian 邊緣偵測

```
import skimage  
import cv2  
import numpy as np  
from google.colab.patches import cv2_imshow  
  
# 讀取範例影像並轉灰階  
image = skimage.data.chelsea() # 彩色  
(RGB)  
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY) # 灰階
```

```
# 先做一點模糊，減少雜訊
gray_blur = cv2.GaussianBlur(gray, (3, 3), 0)

# Laplacian 邊緣偵測
lap = cv2.Laplacian(gray_blur, ddepth=cv2.CV_64F,
                    ksize=3)

# 取絕對值並轉回 8-bit，方便顯示
lap_abs = cv2.convertScaleAbs(lap)

# 顯示結果
print("Laplacian edges")
cv2_imshow(lap_abs)
```



1-5、Sobel X 軸邊緣偵測

```
import skimage
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# 讀取範例影像並轉成灰階
image = skimage.data.chelsea()
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
```

```
# 先稍微模糊，減少雜訊
gray_blur = cv2.GaussianBlur(gray, (3, 3), 0)

# Sobel X 邊緣偵測 (dx=1, dy=0 代表只算 x 方向導數)
sobel_x = cv2.Sobel(src=gray_blur, ddepth=cv2.CV_64F,
                    dx=1, dy=0, ksize=3)

# 取絕對值並轉回 8-bit，方便顯示
sobel_x_abs = cv2.convertScaleAbs(sobel_x)

# 顯示結果
print("Sobel X edge detection")
cv2_imshow(sobel_x_abs)
```



1-6、 Sobel Y 軸邊緣偵測

```
import skimage
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# 讀取範例影像並轉成灰階
image = skimage.data.chelsea()
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
```

```
# 先稍微模糊，減少雜訊
gray_blur = cv2.GaussianBlur(gray, (3, 3), 0)

# Sobel Y 邊緣偵測 (dx=0, dy=1 代表只算 Y 方向導數)
sobel_y = cv2.Sobel(src=gray_blur, ddepth=cv2.CV_64F,
                    dx=0, dy=1, ksize=3)

# 取絕對值並轉回 8-bit，方便顯示
sobel_y_abs = cv2.convertScaleAbs(sobel_y)

# 顯示結果
print("Sobel Y edge detection")
cv2.imshow(sobel_y_abs)
```



Q2. 多元分類練習：請參照下面連結網頁的說明，試以「Rice Image」數據集建立卷積神經網路，來預測（識別）5 種稻米圖片種類：Arborio、Basmati、Ipsala、Jasmine、Karacadag。

程式碼如下：

```
# 匯入資料集
```

```
import os
from tensorflow import keras
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
sns.set(style="darkgrid", color_codes = True)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten

import warnings
warnings.filterwarnings("ignore")
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
```

```
data_path = '/kaggle/input/rice-image-dataset/Rice_Image_Dataset'
```

```
images = []    # 用來存放所有圖片的路徑
labels = []    # 用來存放對應的標籤（子資料夾名稱）
```

```
# 走訪資料夾下的每一個子資料夾（每一類的米種）
for subfolder in os.listdir(data_path):
    subfolder_path = os.path.join(data_path, subfolder)
    if not os.path.isdir(subfolder_path):    # 如果不是資料夾就跳過（避免讀到
多餘檔案）
        continue
    # 走訪該子資料夾中的每一張圖片
    for image_filename in os.listdir(subfolder_path):
        image_path = os.path.join(subfolder_path, image_filename)    # 組合出圖
```


片完整路徑

```
images.append(image_path)    # 加入圖片路徑
labels.append(subfolder)     # 用子資料夾名稱當成圖片的標籤
```

建立一個 DataFrame，把圖片路徑和標籤對應起來

```
df = pd.DataFrame({'image': images, 'label': labels})
```

#顯示 DataFrame 的前五列資料

```
df.head()
```

	image	label
0	/kaggle/input/rice-image-dataset/Rice_Image_Da...	Karacadag
1	/kaggle/input/rice-image-dataset/Rice_Image_Da...	Karacadag
2	/kaggle/input/rice-image-dataset/Rice_Image_Da...	Karacadag
3	/kaggle/input/rice-image-dataset/Rice_Image_Da...	Karacadag
4	/kaggle/input/rice-image-dataset/Rice_Image_Da...	Karacadag

將資料分成訓練集與測試集（圖片路徑與標籤），其中 20% 保留給測試，
random_state 確保每次切分一致

```
x_train, x_test, y_train, y_test = train_test_split(df['image'], df['label'], test_size=0.2,  
random_state=42)
```

建立訓練集的 DataFrame，方便後續資料存取

```
df_train = pd.DataFrame({'image': x_train, 'label': y_train})
```

建立測試集的 DataFrame

```
df_test = pd.DataFrame({'image': x_test, 'label': y_test})
```

用 LabelEncoder 將標籤（文字）轉換成數值，用於模型訓練

```
encoder = LabelEncoder()
```

```
y_train = encoder.fit_transform(y_train)    # 訓練集標籤轉成數值
```

```
y_test = encoder.transform(y_test)          # 測試集標籤也一起轉換
```

定義訓練圖片的高寬及批次大小

IMG_WIDTH = 100

IMG_HEIGHT = 100

BATCH_SIZE = 32

建立資料增強器，設定圖片隨機旋轉、平移、縮放等，提高模型泛化能力

train_datagen = ImageDataGenerator(

rotation_range=45,

width_shift_range=0.2,

height_shift_range=0.2,

shear_range=0.2,

zoom_range=0.2,

rescale=1.0/255,

horizontal_flip=True,

fill_mode='nearest'

)

用資料增強器產生訓練用批次，每次讀入並轉換一批圖片

train_generator = train_datagen.flow_from_dataframe(

df_train,

x_col='image',

y_col='label',

target_size=(IMG_WIDTH, IMG_HEIGHT), # 圖片縮放成統一大小

batch_size=BATCH_SIZE, # 批次大小

class_mode='categorical',

shuffle=True

)

Found 60000 validated image filenames belonging to 5 classes.

用同樣方式建立測試資料的生成器，但是不打亂順序

test_generator=train_datagen.flow_from_dataframe(

df_test,

x_col='image',

y_col='label',

```
        target_size=(IMG_WIDTH, IMG_HEIGHT),
        batch_size=BATCH_SIZE,
        class_mode='categorical',
        shuffle=False
    )
    Found 15000 validated image filenames belonging to 5 classes.
```

從資料夾自動產生驗證集（validation set）的生成器，隨機打亂、批次處理
（subset='validation' 需預先設定 validation_split）

```
validation_generator = train_datagen.flow_from_directory(data_path,
    target_size=(IMG_WIDTH, IMG_HEIGHT),
    batch_size=512,
    class_mode='categorical',
    shuffle=True,
    subset='validation')

Found 0 images belonging to 5 classes.
```

將生成器中的類別索引(class_indices)反轉，變成：數字標籤對應分類名稱
labels = {value: key for key, value in train_generator.class_indices.items()}

用於顯示所有類別的編號與名稱對應關係

```
for key, value in labels.items():
    print(f"{key} : {value}")
```

```
0 : Arborio
1 : Basmati
2 : Ipsala
3 : Jasmine
4 : Karacadag
```

設定每個類別要顯示幾張圖片，每列顯示一種標籤

```
images_per_class = 5
unique_labels = df['label'].unique()
```

```

n_classes = len(unique_labels)

# 建立多個子圖，figsize 控制整體版面尺寸
fig, axes = plt.subplots(n_classes, images_per_class + 1, figsize=(images_per_class * 3
+ 2, n_classes * 3))

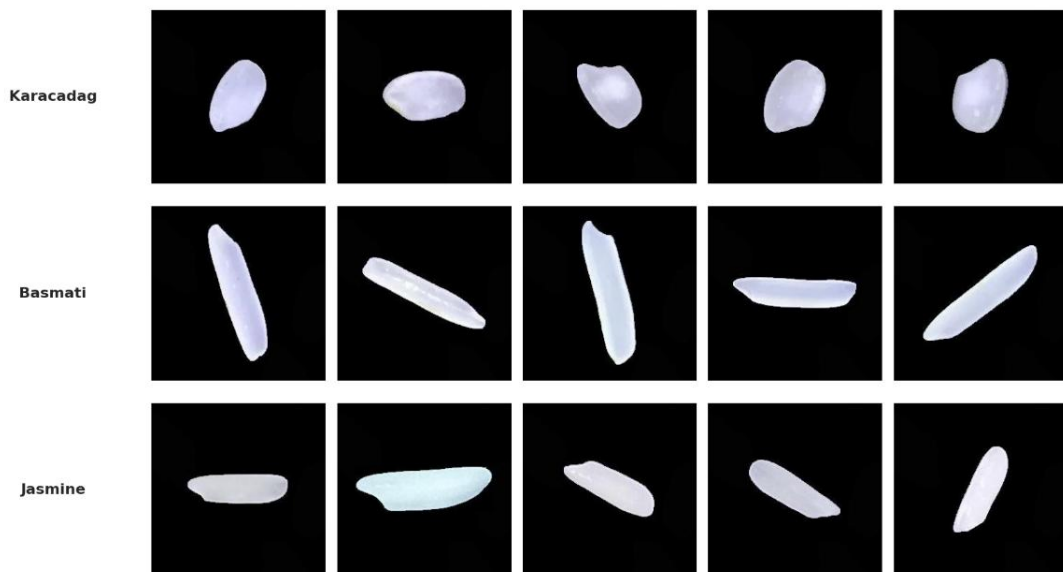
for row, category in enumerate(unique_labels):
    # 篩選每個標籤的前幾張圖片
    filepaths = df[df['label'] == category]['image'].values[:images_per_class]

    # 在每一列第一格顯示類別名稱
    axes[row, 0].text(0.5, 0.5, category, fontsize=16, fontweight='bold', ha='center',
va='center')
    axes[row, 0].axis('off')

    # 依序將選出的圖片讀入並顯示在當列
    for col, filepath in enumerate(filepaths):
        axes[row, col + 1].imshow(mpimg.imread(filepath))
        axes[row, col + 1].axis('off')

plt.tight_layout()    # 調整子圖間距避免重疊
plt.show()           # 顯示整個圖像陣列

```





```
input_shape=(100,100,3)
```

```
model = Sequential([
```

```
    Conv2D(32, (3,3), activation='relu', input_shape=input_shape),
```

```
# 第一個卷積層：提取局部特徵
```

```
    MaxPooling2D((2,2)),
```

```
#第一個池化層：降低特徵圖尺寸
```

```
    Conv2D(64, (3,3), activation='relu'),
```

```
# 第二個卷積層：提取更高階特徵
```

```
    MaxPooling2D((2,2)),
```

```
# 第二個池化層：再次降低尺寸
```

```
    Flatten(),
```

```
# 攤平成一維向量
```

```
    Dense(128, activation='relu'),
```

```
# 全連接層：進行特徵組合
```

```
    Dense(5, activation='softmax')
```

```
# 輸出層：多類別預測（5 類）
```

```
])
```

```
#設定模型訓練
```

```
model.compile(loss='categorical_crossentropy', optimizer='adam',
```

```
metrics=['accuracy'])
```

```
#列出神經網路模型的結構
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 98, 98, 32)	896
max_pooling2d (MaxPooling2D)	(None, 49, 49, 32)	0
conv2d_1 (Conv2D)	(None, 47, 47, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 23, 23, 64)	0
flatten (Flatten)	(None, 33856)	0
dense (Dense)	(None, 128)	4,333,696
dense_1 (Dense)	(None, 5)	645

Total params: 4,353,733 (16.61 MB)

Trainable params: 4,353,733 (16.61 MB)

Non-trainable params: 0 (0.00 B)

設定回呼函數 ModelCheckpoint，用來保存訓練過程中表現最好的模型

```
checkpoint = ModelCheckpoint(  
    filepath=new_filepath,      # 儲存檔案路徑  
    monitor='val_accuracy',    # 監控驗證集準確率  
    verbose=1,                 # 出現訊息（模型有被儲存時會顯示）  
    save_best_only=True,       # 只保存表現最好的（驗證準確率最高的）  
    mode='max'                 # 監控指標越高越好（準確率用 max）  
)
```

開始訓練神經網路模型

```
history=model.fit(  
    train_generator,  
    epochs=5,  
    batch_size=4,  
    validation_data=test_generator,  
    callbacks=[checkpoint]  
)
```

```

Epoch 1/5
1875/1875 ————— 0s 285ms/step - accuracy: 0.7480 - loss: 0.6067
Epoch 1: val_accuracy improved from -inf to 0.93653, saving model to my_Best_Model_v2.keras
1875/1875 ————— 675s 359ms/step - accuracy: 0.7481 - loss: 0.6065 - val_accuracy: 0.9365 - val_loss: 0.1674
Epoch 2/5
1875/1875 ————— 0s 275ms/step - accuracy: 0.9409 - loss: 0.1602
Epoch 2: val_accuracy improved from 0.93653 to 0.94680, saving model to my_Best_Model_v2.keras
1875/1875 ————— 602s 321ms/step - accuracy: 0.9409 - loss: 0.1602 - val_accuracy: 0.9468 - val_loss: 0.1414
Epoch 3/5
1875/1875 ————— 0s 269ms/step - accuracy: 0.9594 - loss: 0.1159
Epoch 3: val_accuracy improved from 0.94680 to 0.97867, saving model to my_Best_Model_v2.keras
1875/1875 ————— 594s 317ms/step - accuracy: 0.9594 - loss: 0.1159 - val_accuracy: 0.9787 - val_loss: 0.0653
Epoch 4/5
1875/1875 ————— 0s 274ms/step - accuracy: 0.9706 - loss: 0.0827
Epoch 4: val_accuracy did not improve from 0.97867
1875/1875 ————— 596s 318ms/step - accuracy: 0.9706 - loss: 0.0827 - val_accuracy: 0.9674 - val_loss: 0.0939
Epoch 5/5
1875/1875 ————— 0s 270ms/step - accuracy: 0.9721 - loss: 0.0795
Epoch 5: val_accuracy improved from 0.97867 to 0.98173, saving model to my_Best_Model_v2.keras
1875/1875 ————— 592s 316ms/step - accuracy: 0.9721 - loss: 0.0795 - val_accuracy: 0.9817 - val_loss: 0.0563

```

模型訓練過程中的準確率和損失（loss）隨時間變化狀況

```
import matplotlib.pyplot as plt
```

```
plt.style.use("default")
```

```

train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
epochs = range(1, len(train_acc) + 1)

```

```

plt.figure(figsize=(8, 5), facecolor='white')
plt.plot(epochs, train_acc, label='Train')
plt.plot(epochs, val_acc, label='Validation')
plt.title('Train vs Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.grid(True, color='gray', linestyle='--', linewidth=0.5)
plt.legend()
plt.show()

```

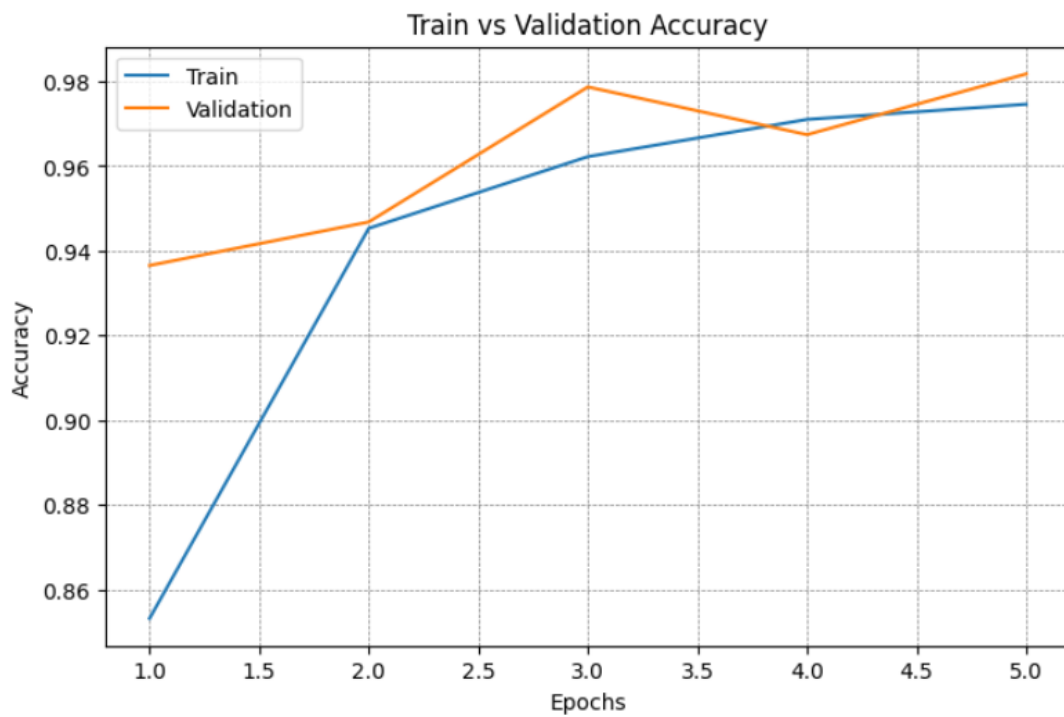
```

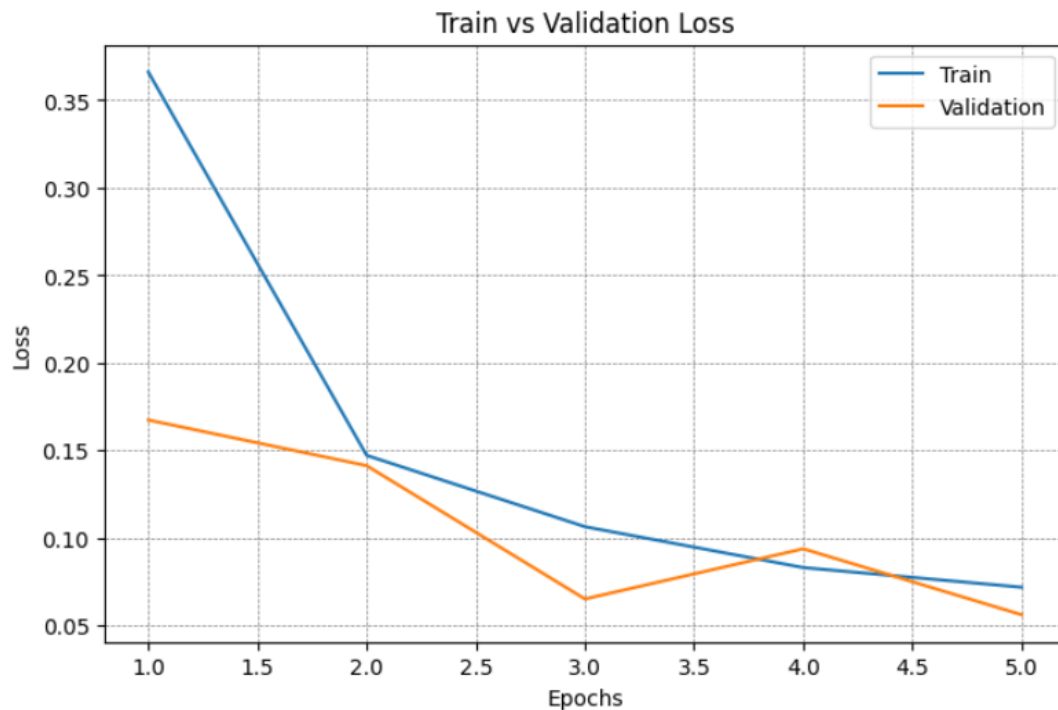
train_loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(1, len(train_loss) + 1)

```

```
plt.figure(figsize=(8, 5), facecolor='white')
```

```
plt.plot(epochs, train_loss, label='Train')
plt.plot(epochs, val_loss, label='Validation')
plt.title('Train vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.grid(True, color='gray', linestyle='--', linewidth=0.5)
plt.legend()
plt.show()
```





把先前訓練並儲存的神經網路模型從檔案讀回來，以便繼續使用

```
from keras.models import load_model  
cnn=load_model('my_Best_Model_v2.keras')
```

預測&混淆矩陣

```
cnn.build((None, 100, 100, 3))
```

```
from sklearn.metrics import confusion_matrix, classification_report
```

```
y_pred_prob = cnn.predict(test_generator, verbose=0)
```

```
y_pred = np.argmax(y_pred_prob, axis=1)
```

```
y_true = test_generator.classes
```

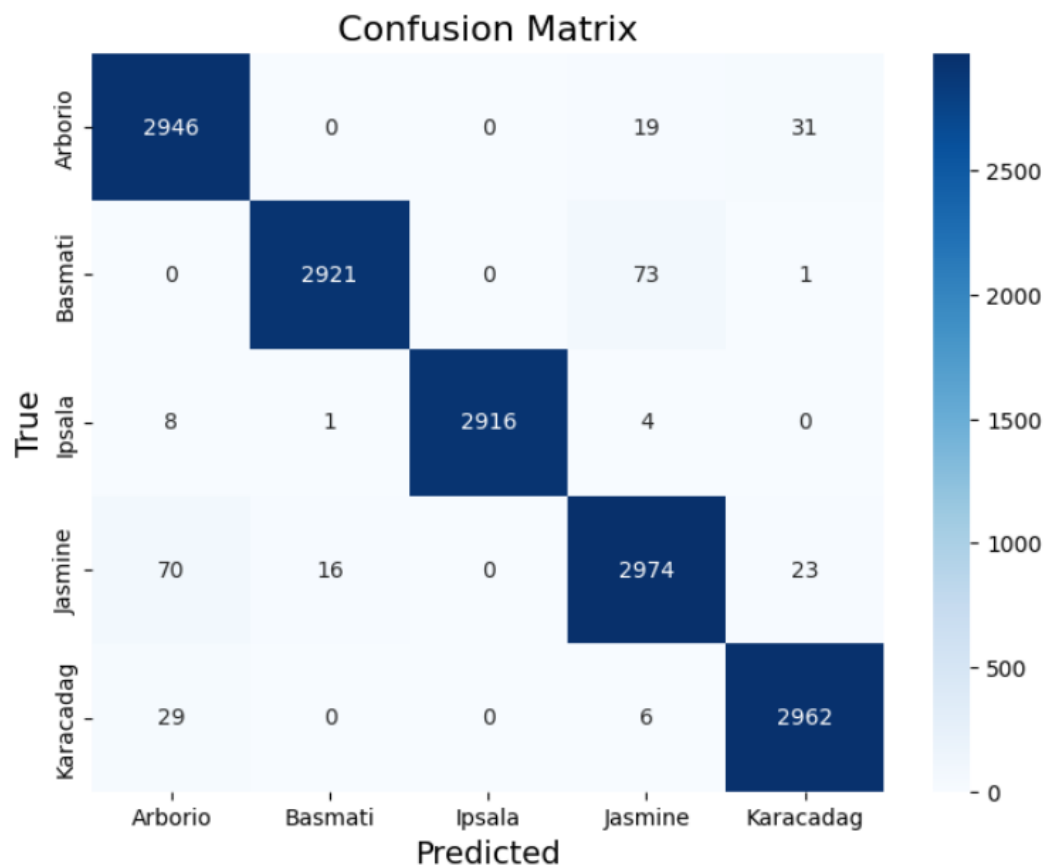
```
cm = confusion_matrix(y_true, y_pred)
```

```
plt.figure(figsize=(8,6))
```

```
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
             xticklabels=list(train_generator.class_indices.keys()),
             yticklabels=list(train_generator.class_indices.keys()))
plt.title("Confusion Matrix", fontsize=16)
plt.xlabel("Predicted", fontsize=14)
plt.ylabel("True", fontsize=14)
plt.show()
```

分類報告

```
print("\n=== Classification Report ===\n")
print(classification_report(y_true, y_pred,
                             target_names=list(train_generator.class_indices.keys())))
```



```

=== Classification Report ===

              precision    recall  f1-score   support

   Arborio      0.96      0.98      0.97     2996
   Basmati      0.99      0.98      0.98     2995
   Ipsala       1.00      1.00      1.00     2929
   Jasmine      0.97      0.96      0.97     3083
   Karacadag     0.98      0.99      0.99     2997

 accuracy      0.98      0.98      0.98    15000
  macro avg     0.98      0.98      0.98    15000
weighted avg     0.98      0.98      0.98    15000

```

Q3. 多元分類練習：請參照下面連結網頁的說明，試以「Sign Language Digits」數據集建立卷積神經網路，來預測（識別）數字 0~9 的手語圖片

程式碼如下：

```

#匯入資料庫&資料集
import numpy as np
from sklearn.model_selection import train_test_split
import pandas as pd
import matplotlib.pyplot as plt

data_dir = '/kaggle/input/sign-language-digits-dataset'
X = np.load(f'{data_dir}/X.npy').astype('float32') / 255.0 # 載入影像特徵並將像素
值標準化到 0~1
Y = np.load(f'{data_dir}/Y.npy') # 載入標籤

if X.ndim == 3:
    # 如果 X 是三維陣列（高、寬、數量）
    X = np.expand_dims(X, axis=-1) # 增加一個新的維度（轉為四維）

```

```
# 將 Y 這個（可能是 one-hot 編碼的標籤）轉換成「數字類別」的形式
Y_int = np.argmax(Y, axis=1)
```

```
#將數值類別標籤轉成文字標籤
```

```
label_map = {0:"Zero",1:"One",2:"Two",3:"Three",4:"Four",
              5:"Five",6:"Six",7:"Seven",8:"Eight",9:"Nine"}
```

```
Y_decoded = np.array([label_map[i] for i in Y_int])
```

```
print(Y_decoded[:10])
```

```
['Zero' 'Zero' 'Zero' 'Zero' 'Zero' 'Zero' 'Zero' 'Zero' 'Zero' 'Zero']
```

```
#把一批影像的基本資訊和標籤整理成一個 pandas DataFrame 表格
```

```
df = pd.DataFrame({
    'label': Y_decoded,
    'height': [img.shape[0] for img in X],
    'width': [img.shape[1] for img in X],
    'channels': [img.shape[2] for img in X],
    'mean_pixel': [img.mean() for img in X],
    'std_pixel': [img.std() for img in X],
    'min_pixel': [img.min() for img in X],
    'max_pixel': [img.max() for img in X]
})
```

```
df
```

	label	height	width	channels	mean_pixel	std_pixel	min_pixel	max_pixel
0	Zero	64	64	1	0.002634	0.000559	0.000754	0.003799
1	Zero	64	64	1	0.002657	0.000636	0.000677	0.003799
2	Zero	64	64	1	0.002698	0.000577	0.000677	0.003768
3	Zero	64	64	1	0.002617	0.000603	0.000769	0.003737
4	Zero	64	64	1	0.002761	0.000578	0.000569	0.003799
...	...	...	...	...	...	...	...	...
2057	Nine	64	64	1	0.002456	0.000623	0.000446	0.003752
2058	Nine	64	64	1	0.002639	0.000593	0.000338	0.003814
2059	Nine	64	64	1	0.002525	0.000573	0.000815	0.003722
2060	Nine	64	64	1	0.002655	0.000601	0.000661	0.003768
2061	Nine	64	64	1	0.003043	0.000648	0.000477	0.003860

2062 rows × 8 columns

#顯示 DataFrame 的前五列資料

df.head()

	label	height	width	channels	mean_pixel	std_pixel	min_pixel	max_pixel
0	Zero	64	64	1	0.002634	0.000559	0.000754	0.003799
1	Zero	64	64	1	0.002657	0.000636	0.000677	0.003799
2	Zero	64	64	1	0.002698	0.000577	0.000677	0.003768
3	Zero	64	64	1	0.002617	0.000603	0.000769	0.003737
4	Zero	64	64	1	0.002761	0.000578	0.000569	0.003799

對 DataFrame 快速產生一組統計摘要

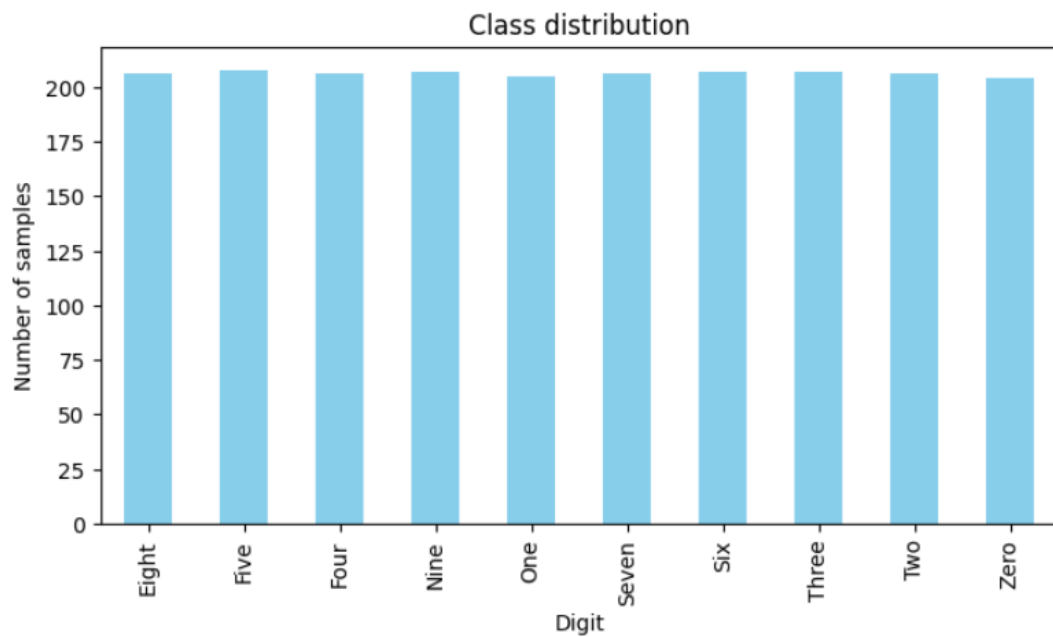
df.describe()

	height	width	channels	mean_pixel	std_pixel	min_pixel	max_pixel
count	2062.0	2062.0	2062.0	2062.000000	2062.000000	2062.000000	2062.000000
mean	64.0	64.0	1.0	0.002670	0.000616	0.000599	0.003777
std	0.0	0.0	0.0	0.000150	0.000050	0.000209	0.000060
min	64.0	64.0	1.0	0.002153	0.000488	0.000015	0.003137
25%	64.0	64.0	1.0	0.002593	0.000582	0.000461	0.003752
50%	64.0	64.0	1.0	0.002656	0.000608	0.000631	0.003783
75%	64.0	64.0	1.0	0.002712	0.000639	0.000754	0.003814
max	64.0	64.0	1.0	0.003287	0.000856	0.001153	0.003922

```
# 類別分布長條圖
```

```
import matplotlib.pyplot as plt
```

```
df['label'].value_counts().sort_index().plot(  
    kind='bar', figsize=(8,4), color='skyblue', title='Class distribution'  
)  
plt.xlabel('Digit')  
plt.ylabel('Number of samples')  
plt.show()
```



```
# 隨機挑選 9 張影像並以 9 宮格的方式顯示出來
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
num_samples = X.shape[0]
```

```
random_indices = np.random.choice(num_samples, size=9, replace=False)
```

```
plt.figure(figsize=(8,8))
```

```
for i, idx in enumerate(random_indices):
```

```
    plt.subplot(3,3,i+1)
```

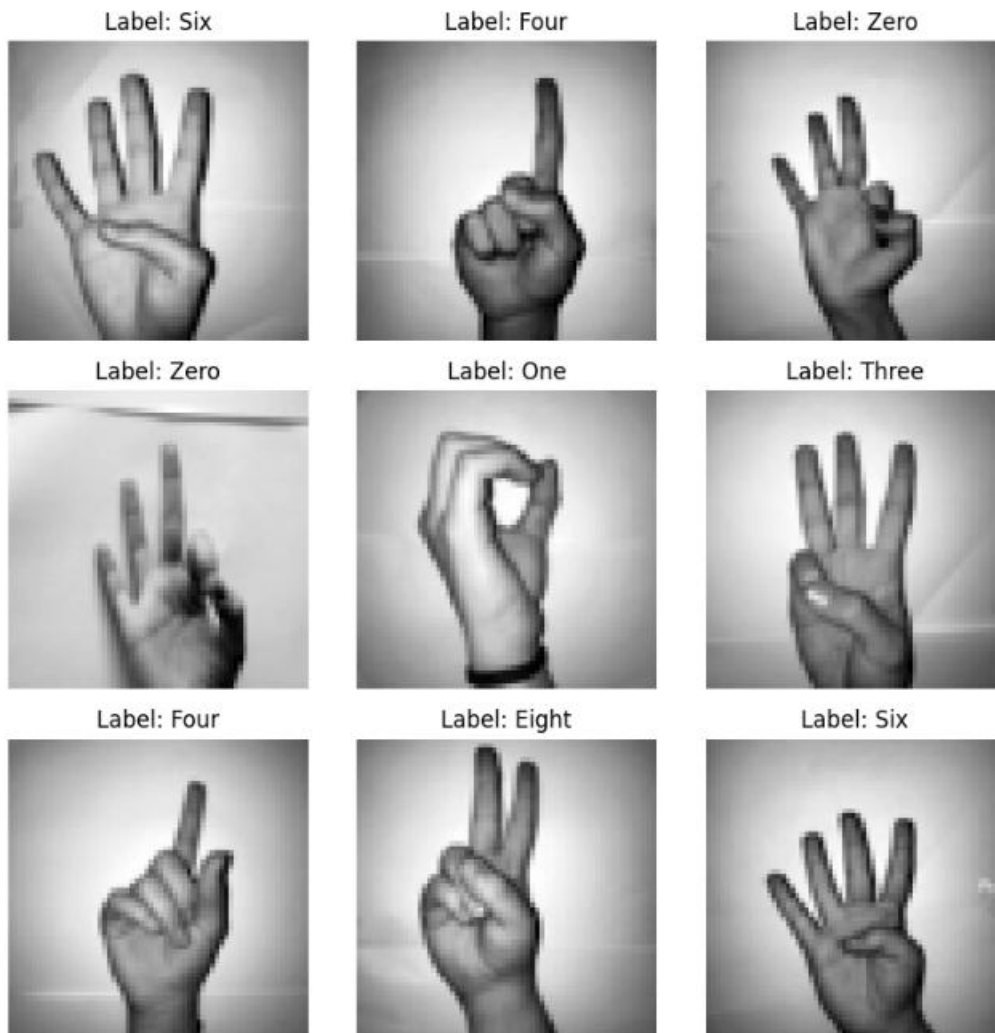
```
    img = X[idx]
```

```
    if img.shape[2] == 1:
```

```
        img = img.squeeze()
```

```
plt.imshow(img, cmap='gray')
plt.title(f'Label: {Y_decoded[idx]}')
plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```



資料集的分割和標籤編碼

```
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
```

```
X_train, X_val, Y_train_int, Y_val_int = train_test_split(X, Y_int, test_size=0.2,
random_state=42)
```

```
Y_train = to_categorical(Y_train_int, num_classes=10)
```

```
Y_val = to_categorical(Y_val_int, num_classes=10)
```

```

# 統計訓練集和驗證集每個數字類別（0~9）各自有幾筆資料
print("Train class distribution:", np.bincount(Y_train_int))
print("Val class distribution:", np.bincount(Y_val_int))

Train class distribution: [164 157 156 172 174 170 166 168 161 161]
Val class distribution: [40 48 50 35 32 38 41 38 45 46]


import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
BatchNormalization

from sklearn.metrics import classification_report


# 建立 CNN 模型架構
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(64, 64, 1)),
    BatchNormalization(),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Conv2D(64, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Conv2D(128, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Flatten(),

```



```

        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')
    ])

# 編譯模型
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

from tensorflow.keras.callbacks import Callback

# 提早停止自訂 Callback
class StopOnValAccuracy(Callback):
    def __init__(self, threshold=0.95):
        super().__init__()
        self.threshold = threshold

    def on_epoch_end(self, epoch, logs=None):
        val_acc = logs.get('val_accuracy')
        if val_acc is not None and val_acc > self.threshold:
            print(f"\nStopping training: val_accuracy reached {val_acc:.4f}")
            self.model.stop_training = True

stop_callback = StopOnValAccuracy(threshold=0.95)

# 模型訓練
history = model.fit(
    X_train, Y_train,
    validation_data=(X_val, Y_val),
    epochs=50,
    batch_size=32,
    callbacks=[stop_callback]
)

```

```

Epoch 1/50
52/52 ————— 15s 196ms/step - accuracy: 0.1180 - loss: 3.4672 - val_accuracy: 0.0775 - val_loss: 2.6722
Epoch 2/50
52/52 ————— 9s 177ms/step - accuracy: 0.1428 - loss: 2.2468 - val_accuracy: 0.0775 - val_loss: 3.8297
Epoch 3/50
52/52 ————— 9s 176ms/step - accuracy: 0.1571 - loss: 2.2200 - val_accuracy: 0.0775 - val_loss: 5.4737
Epoch 4/50
52/52 ————— 9s 178ms/step - accuracy: 0.2166 - loss: 2.0718 - val_accuracy: 0.0775 - val_loss: 7.7597
Epoch 5/50
52/52 ————— 8s 152ms/step - accuracy: 0.3609 - loss: 1.8146 - val_accuracy: 0.0920 - val_loss: 10.0941
Epoch 6/50
52/52 ————— 7s 143ms/step - accuracy: 0.4156 - loss: 1.5967 - val_accuracy: 0.0920 - val_loss: 12.4755
Epoch 7/50
52/52 ————— 10s 146ms/step - accuracy: 0.4216 - loss: 1.5004 - val_accuracy: 0.0920 - val_loss: 13.5421
Epoch 8/50
52/52 ————— 10s 145ms/step - accuracy: 0.4680 - loss: 1.3625 - val_accuracy: 0.0920 - val_loss: 15.0695
Epoch 9/50
52/52 ————— 8s 145ms/step - accuracy: 0.5158 - loss: 1.2922 - val_accuracy: 0.0920 - val_loss: 13.9800
Epoch 10/50
52/52 ————— 7s 137ms/step - accuracy: 0.5448 - loss: 1.1387 - val_accuracy: 0.0920 - val_loss: 14.2466
Epoch 11/50
52/52 ————— 7s 136ms/step - accuracy: 0.5504 - loss: 1.1307 - val_accuracy: 0.2978 - val_loss: 2.2476
Epoch 12/50
52/52 ————— 7s 142ms/step - accuracy: 0.6188 - loss: 0.9671 - val_accuracy: 0.2954 - val_loss: 2.1757
Epoch 13/50
52/52 ————— 7s 141ms/step - accuracy: 0.6636 - loss: 0.8272 - val_accuracy: 0.4697 - val_loss: 1.4258
Epoch 14/50
52/52 ————— 8s 147ms/step - accuracy: 0.6683 - loss: 0.8332 - val_accuracy: 0.2736 - val_loss: 1.8042
Epoch 15/50
52/52 ————— 8s 146ms/step - accuracy: 0.7130 - loss: 0.7152 - val_accuracy: 0.1719 - val_loss: 4.9187
Epoch 16/50
52/52 ————— 8s 145ms/step - accuracy: 0.7238 - loss: 0.7158 - val_accuracy: 0.1937 - val_loss: 5.1325
Epoch 17/50
52/52 ————— 10s 150ms/step - accuracy: 0.7305 - loss: 0.7135 - val_accuracy: 0.0920 - val_loss: 37.4941
Epoch 18/50
52/52 ————— 8s 151ms/step - accuracy: 0.7670 - loss: 0.6007 - val_accuracy: 0.2349 - val_loss: 2.5958
Epoch 19/50
52/52 ————— 7s 143ms/step - accuracy: 0.7369 - loss: 0.6568 - val_accuracy: 0.0944 - val_loss: 4.2489
Epoch 20/50
52/52 ————— 7s 143ms/step - accuracy: 0.7635 - loss: 0.5500 - val_accuracy: 0.2350 - val_loss: 2.3470
Epoch 21/50
52/52 ————— 7s 143ms/step - accuracy: 0.7635 - loss: 0.5500 - val_accuracy: 0.2350 - val_loss: 2.3470
Epoch 22/50
52/52 ————— 7s 143ms/step - accuracy: 0.7635 - loss: 0.5500 - val_accuracy: 0.2350 - val_loss: 2.3470
Epoch 23/50
52/52 ————— 7s 143ms/step - accuracy: 0.7635 - loss: 0.5500 - val_accuracy: 0.2350 - val_loss: 2.3470
Epoch 24/50
52/52 ————— 7s 143ms/step - accuracy: 0.7635 - loss: 0.5500 - val_accuracy: 0.2350 - val_loss: 2.3470
Epoch 25/50
52/52 ————— 8s 146ms/step - accuracy: 0.8254 - loss: 0.4660 - val_accuracy: 0.6513 - val_loss: 0.8357
Epoch 26/50
52/52 ————— 7s 142ms/step - accuracy: 0.8332 - loss: 0.4419 - val_accuracy: 0.8136 - val_loss: 0.6201
Epoch 27/50
52/52 ————— 8s 147ms/step - accuracy: 0.8309 - loss: 0.4193 - val_accuracy: 0.3995 - val_loss: 1.6689
Epoch 28/50
52/52 ————— 7s 144ms/step - accuracy: 0.8383 - loss: 0.4347 - val_accuracy: 0.4479 - val_loss: 1.7705
Epoch 29/50
52/52 ————— 8s 149ms/step - accuracy: 0.8683 - loss: 0.3477 - val_accuracy: 0.6513 - val_loss: 0.9133
Epoch 30/50
52/52 ————— 8s 146ms/step - accuracy: 0.8485 - loss: 0.3934 - val_accuracy: 0.5036 - val_loss: 1.2894
Epoch 31/50
52/52 ————— 8s 152ms/step - accuracy: 0.8572 - loss: 0.3689 - val_accuracy: 0.3390 - val_loss: 1.6604
Epoch 32/50
52/52 ————— 8s 150ms/step - accuracy: 0.8550 - loss: 0.3518 - val_accuracy: 0.1477 - val_loss: 14.6791
Epoch 33/50
52/52 ————— 10s 149ms/step - accuracy: 0.8674 - loss: 0.3481 - val_accuracy: 0.1792 - val_loss: 12.5151
Epoch 34/50
52/52 ————— 8s 147ms/step - accuracy: 0.8545 - loss: 0.3355 - val_accuracy: 0.3390 - val_loss: 10.5321
Epoch 35/50
52/52 ————— 8s 146ms/step - accuracy: 0.8647 - loss: 0.3497 - val_accuracy: 0.7554 - val_loss: 0.9320
Epoch 36/50
52/52 ————— 7s 142ms/step - accuracy: 0.8810 - loss: 0.3006 - val_accuracy: 0.9225 - val_loss: 0.2652
Epoch 37/50
52/52 ————— 11s 147ms/step - accuracy: 0.8861 - loss: 0.2796 - val_accuracy: 0.3293 - val_loss: 10.5198
Epoch 38/50
52/52 ————— 7s 142ms/step - accuracy: 0.8977 - loss: 0.3090 - val_accuracy: 0.9104 - val_loss: 0.3046
Epoch 39/50
52/52 ————— 0s 136ms/step - accuracy: 0.8825 - loss: 0.2807
Stopping training: val_accuracy reached 0.9540
52/52 ————— 7s 143ms/step - accuracy: 0.8827 - loss: 0.2806 - val_accuracy: 0.9540 - val_loss: 0.1491

```

將深度學習模型訓練過程中「準確率」和「損失」的變化用圖形顯示出來

```
import matplotlib.pyplot as plt
```

```
accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(1, len(accracy) + 1)
```

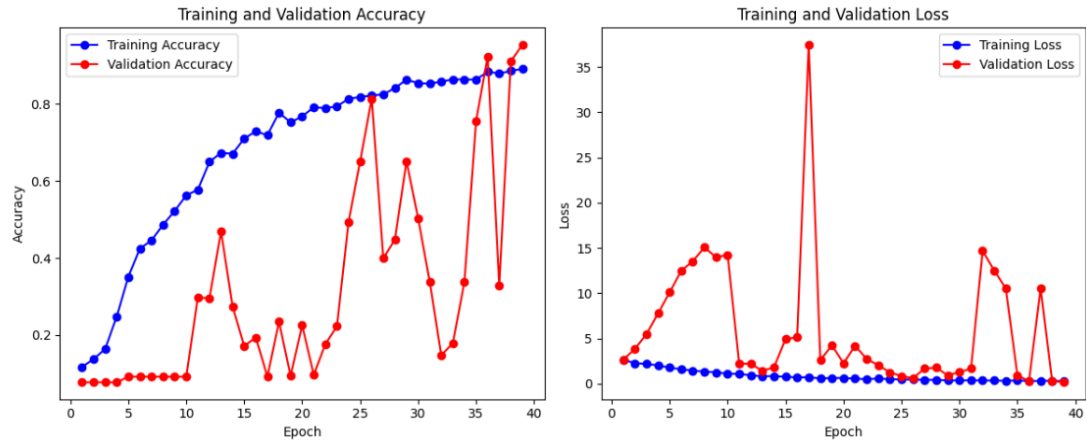
```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.plot(epochs, accuracy, 'bo-', label='Training Accuracy')
plt.plot(epochs, val_accuracy, 'ro-', label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
# Plot loss
```

```
plt.subplot(1, 2, 2)
plt.plot(epochs, loss, 'bo-', label='Training Loss')
plt.plot(epochs, val_loss, 'ro-', label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```



```
from sklearn.metrics import confusion_matrix
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
#把 one-hot 轉成類別標籤
```

```
Y_train_true = np.argmax(Y_train, axis=1)
```

```
Y_train_pred = np.argmax(model.predict(X_train), axis=1)
```

```
Y_val_true = np.argmax(Y_val, axis=1)
```

```
Y_val_pred = np.argmax(model.predict(X_val), axis=1)
```

```
#計算混淆矩陣
```

```
plt.figure(figsize=(8, 6))
```

```
cm_train = confusion_matrix(Y_train_true, Y_train_pred)
```

```
sns.heatmap(cm_train, annot=True, fmt='d', cmap='Greens', xticklabels=range(10),  
yticklabels=range(10))
```

```
plt.title('Confusion Matrix - Training Set')
```

```
plt.xlabel('Predicted Label')
```

```
plt.ylabel('True Label')
```

```
plt.show()
```

```
#把混淆矩陣畫成圖
```

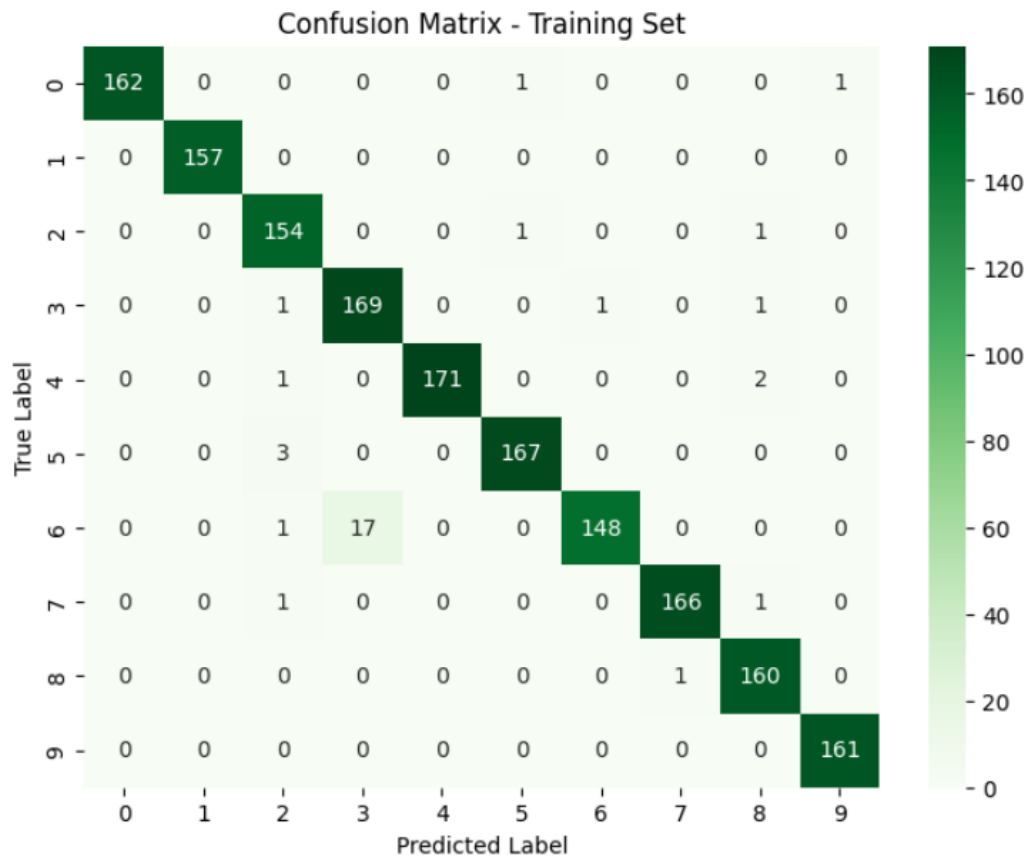
```
plt.figure(figsize=(8, 6))
```

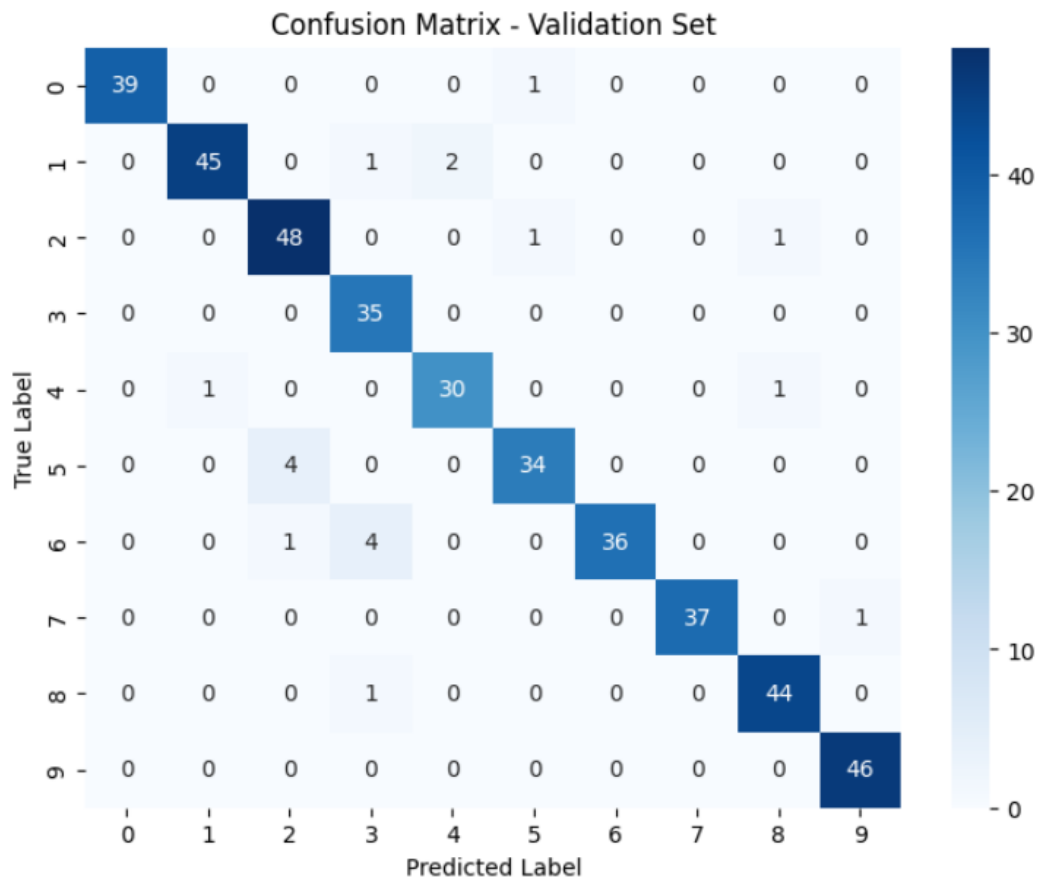
```
cm_val = confusion_matrix(Y_val_true, Y_val_pred)
```

```
sns.heatmap(cm_val, annot=True, fmt='d', cmap='Blues', xticklabels=range(10),  
yticklabels=range(10))
```

```
plt.title('Confusion Matrix - Validation Set')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

52/52 ————— 2s 31ms/step
13/13 ————— 0s 29ms/step





#把模型輸出的機率轉成類別標籤

```
Y_train_pred = np.argmax(model.predict(X_train), axis=1)
```

```
Y_val_pred = np.argmax(model.predict(X_val), axis=1)
```

#用 classification_report 評估訓練集和驗證集的分類效果

```
print("Training set performance:")
```

```
print(classification_report(Y_train_int, Y_train_pred))
```

```
print("Validation set performance:")
```

```
print(classification_report(Y_val_int, Y_val_pred))
```

```

52/52 ————— 2s 29ms/step
13/13 ————— 0s 27ms/step
Training set performance:
      precision    recall  f1-score   support

     0       1.00      0.99      0.99        164
     1       1.00      1.00      1.00        157
     2       0.96      0.99      0.97        156
     3       0.91      0.98      0.94        172
     4       1.00      0.98      0.99        174
     5       0.99      0.98      0.99        170
     6       0.99      0.89      0.94        166
     7       0.99      0.99      0.99        168
     8       0.97      0.99      0.98        161
     9       0.99      1.00      1.00        161

 accuracy          0.98        1649
 macro avg       0.98      0.98      0.98        1649
 weighted avg    0.98      0.98      0.98        1649

```

```

Validation set performance:
      precision    recall  f1-score   support

     0       1.00      0.97      0.99         40
     1       0.98      0.94      0.96         48
     2       0.91      0.96      0.93         50
     3       0.85      1.00      0.92         35
     4       0.94      0.94      0.94         32
     5       0.94      0.89      0.92         38
     6       1.00      0.88      0.94         41
     7       1.00      0.97      0.99         38
     8       0.96      0.98      0.97         45
     9       0.98      1.00      0.99         46

 accuracy          0.95        413
 macro avg       0.96      0.95      0.95        413
 weighted avg    0.96      0.95      0.95        413

```

Q4. 多元分類練習：請參照下面連結網頁的說明，試以「**Brain Tumor MRI**」數據集建立卷積神經網路，來預測（識別）**4**種腦腫瘤狀況：神經膠質瘤、腦膜瘤、無腫瘤、與腦下垂體瘤。

程式碼如下：

```

#匯入資料庫
import tensorflow as tf

```

```

import keras
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

#匯入資料集
import kagglehub

path = kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-dataset")

print("Path to dataset files:", path)

#訓練集和測試集路徑
train_path='/kaggle/input/brain-tumor-mri-dataset/Training'
test_path='/kaggle/input/brain-tumor-mri-dataset/Testing'

#建立一個 Keras 的 ImageDataGenerator 物件，做影像前處理 + 資料增強
（data augmentation）。每個參數都代表你在訓練時，對原始圖片做哪些「隨機
變形」，讓模型看到更多變化、減少 overfitting

train_data=train_datagen.flow_from_directory(
    shuffle=True,
    directory=train_path,
    target_size=(128,128),
    batch_size=32,
    class_mode='categorical')
Found 5712 images belonging to 4 classes.

#類別名稱對應到數字 label 的對照表
train_data.class_indices

{'glioma': 0, 'meningioma': 1, 'notumor': 2, 'pituitary': 3}

```


#建立一個只做「像素縮放」的 ImageDataGenerator，然後把同一個物件同時指定給 train_datagen 和 test_datagen

```
test_datagen=train_datagen=ImageDataGenerator(rescale=1./255)
```

#用 ImageDataGenerator 從資料夾 test_path 裡「生成一個測試資料的批次資料集」，每次會吐出一批已經縮放到 0-1、大小為 128×128、做 one-hot 標籤的圖片和標籤，用來丟進模型做評估或預測。

```
test_data=train_datagen.flow_from_directory(shuffle=False,  
      directory=test_path,  
      target_size=(128,128),  
      batch_size=32,  
      class_mode='categorical')
```

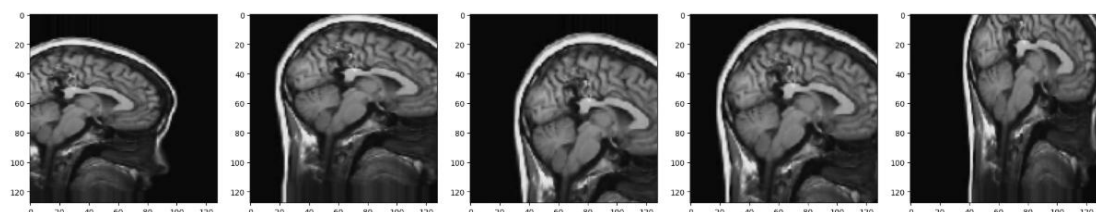
```
Found 1311 images belonging to 4 classes.
```

#在同一張圖裡，把多張圖片排成一排，一次顯示出來方便你查看或比較

```
def plotImages(images_arr):  
    fig, axes = plt.subplots(1, 5, figsize=(20,20))  
    axes = axes.flatten()  
    for img,ax in zip( images_arr, axes):  
        ax.imshow(img)  
    plt.tight_layout()  
    plt.show()
```

#從同一個資料源，複製同一張圖片 5 次放進 list」，然後 plotImages(images) 把這 5 張圖排成一排畫出來

```
images=[train_data[0][0][0] for i in range(5)]  
plotImages(images)
```



```
#建立一個 Sequential 模型
model=keras.models.Sequential()
#卷積+池化：抽特徵、縮小空間
model.add(keras.layers.Conv2D(filters=16,kernel_size=(3,3),activation='relu',input_shape=(128,128,3)))
model.add(keras.layers.MaxPool2D(pool_size=(2,2)))

#後面同樣 pattern 只是 filter 數量變多、特徵越來越抽象
model.add(keras.layers.Conv2D(filters=32,kernel_size=(3,3),activation='relu'))
model.add(keras.layers.MaxPool2D(pool_size=(2,2)))

model.add(keras.layers.Conv2D(filters=64,kernel_size=(3,3),activation='relu'))
model.add(keras.layers.MaxPool2D(pool_size=(2,2)))

model.add(keras.layers.Conv2D(filters=128,kernel_size=(3,3),activation='relu'))
model.add(keras.layers.MaxPool2D(pool_size=(2,2)))

model.add(keras.layers.Conv2D(filters=256,kernel_size=(3,3),activation='relu'))
model.add(keras.layers.MaxPool2D(pool_size=(2,2)))
#Dropout：卷積部分後做正則化
model.add(keras.layers.Dropout(0.5))
#Flatten：把特徵圖攤平成一維向量
model.add(keras.layers.Flatten())
#Dense 全連接層：做分類決策
model.add(keras.layers.Dense(units=128,activation='relu'))
model.add(keras.layers.Dropout(0.1))
model.add(keras.layers.Dense(units=256,activation='relu'))
model.add(keras.layers.Dropout(0.25))
#最後輸出層：4 類 softmax
model.add(keras.layers.Dense(units=4,activation='softmax'))

#印出模型的結構總表
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 16)	448
max_pooling2d (MaxPooling2D)	(None, 63, 63, 16)	0
conv2d_1 (Conv2D)	(None, 61, 61, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 32)	0
conv2d_2 (Conv2D)	(None, 28, 28, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_3 (Conv2D)	(None, 12, 12, 128)	73,856
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 128)	0
conv2d_4 (Conv2D)	(None, 4, 4, 256)	295,168
max_pooling2d_4 (MaxPooling2D)	(None, 2, 2, 256)	0
dropout (Dropout)	(None, 2, 2, 256)	0
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 128)	131,200
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 256)	33,024
dropout_2 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 4)	1,028

Total params: 557,860 (2.13 MB)

Trainable params: 557,860 (2.13 MB)

Non-trainable params: 0 (0.00 B)

#設定訓練規則，用 Adam 來更新權重、用 categorical_crossentropy 當目標函數、用 accuracy 來看表現

```
model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
```

#開始訓練模型

```
history=model.fit(train_data,epochs=40,validation_data=test_data)
```

```

Epoch 1/40
179/179 ————— 120s 642ms/step - accuracy: 0.3849 - loss: 1.2237 - val_accuracy: 0.5385 - val_loss: 1.0673
Epoch 2/40
179/179 ————— 98s 399ms/step - accuracy: 0.5854 - loss: 0.9575 - val_accuracy: 0.4737 - val_loss: 1.1357
Epoch 3/40
179/179 ————— 71s 395ms/step - accuracy: 0.6760 - loss: 0.8103 - val_accuracy: 0.5408 - val_loss: 1.3831
Epoch 4/40
179/179 ————— 70s 392ms/step - accuracy: 0.7093 - loss: 0.7155 - val_accuracy: 0.6270 - val_loss: 0.9834
Epoch 5/40
179/179 ————— 70s 392ms/step - accuracy: 0.7603 - loss: 0.6338 - val_accuracy: 0.7818 - val_loss: 0.5513
Epoch 6/40
179/179 ————— 70s 392ms/step - accuracy: 0.7673 - loss: 0.5961 - val_accuracy: 0.7361 - val_loss: 0.7490
Epoch 7/40
179/179 ————— 71s 393ms/step - accuracy: 0.8058 - loss: 0.5133 - val_accuracy: 0.7307 - val_loss: 0.7586
Epoch 8/40
179/179 ————— 69s 385ms/step - accuracy: 0.8295 - loss: 0.4661 - val_accuracy: 0.7757 - val_loss: 0.5658
Epoch 9/40
179/179 ————— 70s 390ms/step - accuracy: 0.8449 - loss: 0.4084 - val_accuracy: 0.7918 - val_loss: 0.5983
Epoch 10/40
179/179 ————— 71s 393ms/step - accuracy: 0.8607 - loss: 0.3880 - val_accuracy: 0.8360 - val_loss: 0.3998
Epoch 11/40
179/179 ————— 70s 391ms/step - accuracy: 0.8472 - loss: 0.3892 - val_accuracy: 0.8474 - val_loss: 0.3712
Epoch 12/40
179/179 ————— 69s 388ms/step - accuracy: 0.8716 - loss: 0.3451 - val_accuracy: 0.6979 - val_loss: 0.9809
Epoch 13/40
179/179 ————— 82s 385ms/step - accuracy: 0.8908 - loss: 0.2982 - val_accuracy: 0.8505 - val_loss: 0.3850
Epoch 14/40
179/179 ————— 70s 392ms/step - accuracy: 0.8996 - loss: 0.2927 - val_accuracy: 0.8566 - val_loss: 0.3938
Epoch 15/40
179/179 ————— 70s 390ms/step - accuracy: 0.8966 - loss: 0.2839 - val_accuracy: 0.7826 - val_loss: 0.6632
Epoch 16/40
179/179 ————— 70s 390ms/step - accuracy: 0.9046 - loss: 0.2731 - val_accuracy: 0.8322 - val_loss: 0.3989
Epoch 17/40
179/179 ————— 70s 393ms/step - accuracy: 0.9120 - loss: 0.2553 - val_accuracy: 0.9001 - val_loss: 0.2541
Epoch 18/40
179/179 ————— 70s 392ms/step - accuracy: 0.9258 - loss: 0.2207 - val_accuracy: 0.8833 - val_loss: 0.3007
Epoch 19/40
179/179 ————— 70s 391ms/step - accuracy: 0.9340 - loss: 0.1815 - val_accuracy: 0.9375 - val_loss: 0.1757
Epoch 20/40
179/179 ————— 70s 390ms/step - accuracy: 0.9366 - loss: 0.1780 - val_accuracy: 0.9420 - val_loss: 0.1502
Epoch 21/40
179/179 ————— 70s 393ms/step - accuracy: 0.9396 - loss: 0.1533 - val_accuracy: 0.9252 - val_loss: 0.2033
Epoch 22/40
179/179 ————— 71s 393ms/step - accuracy: 0.9528 - loss: 0.1444 - val_accuracy: 0.9458 - val_loss: 0.1419
Epoch 23/40
179/179 ————— 71s 398ms/step - accuracy: 0.9459 - loss: 0.1411 - val_accuracy: 0.9069 - val_loss: 0.2388
Epoch 24/40
179/179 ————— 70s 392ms/step - accuracy: 0.9526 - loss: 0.1423 - val_accuracy: 0.8398 - val_loss: 0.5772
Epoch 25/40
179/179 ————— 70s 390ms/step - accuracy: 0.9442 - loss: 0.1426 - val_accuracy: 0.9649 - val_loss: 0.0985
Epoch 26/40
179/179 ————— 69s 384ms/step - accuracy: 0.9497 - loss: 0.1345 - val_accuracy: 0.9390 - val_loss: 0.1692
Epoch 27/40
179/179 ————— 69s 387ms/step - accuracy: 0.9560 - loss: 0.1317 - val_accuracy: 0.9558 - val_loss: 0.1436
Epoch 28/40
179/179 ————— 70s 388ms/step - accuracy: 0.9597 - loss: 0.1215 - val_accuracy: 0.9237 - val_loss: 0.2072
Epoch 29/40
179/179 ————— 71s 397ms/step - accuracy: 0.9574 - loss: 0.1282 - val_accuracy: 0.9222 - val_loss: 0.1895
Epoch 30/40
179/179 ————— 72s 403ms/step - accuracy: 0.9524 - loss: 0.1412 - val_accuracy: 0.9199 - val_loss: 0.1961
Epoch 31/40
179/179 ————— 71s 395ms/step - accuracy: 0.9548 - loss: 0.1216 - val_accuracy: 0.9367 - val_loss: 0.1799
Epoch 32/40
179/179 ————— 69s 387ms/step - accuracy: 0.9514 - loss: 0.1442 - val_accuracy: 0.9497 - val_loss: 0.1816

```

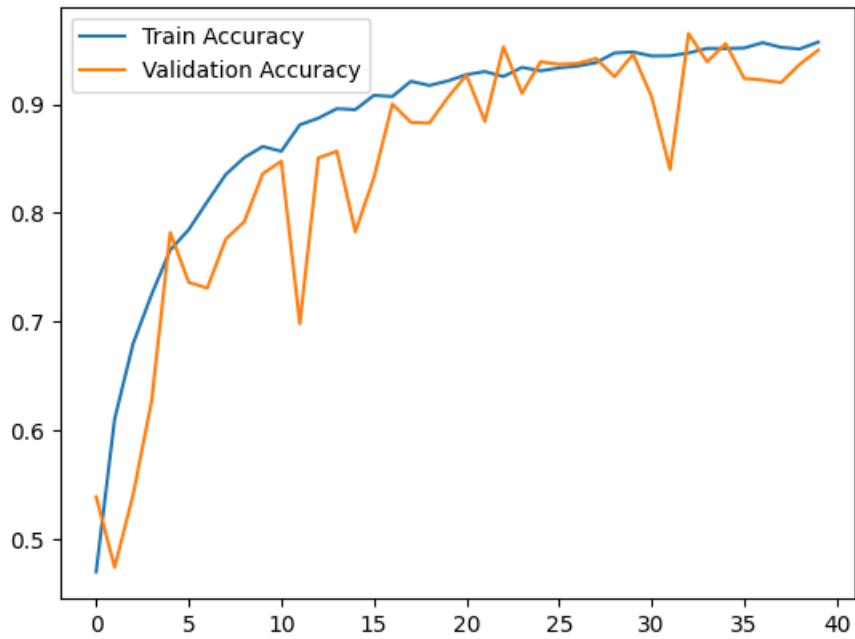
#把訓練過程裡記錄下來的「準確率」拿出來畫成圖

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
```

```
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
```

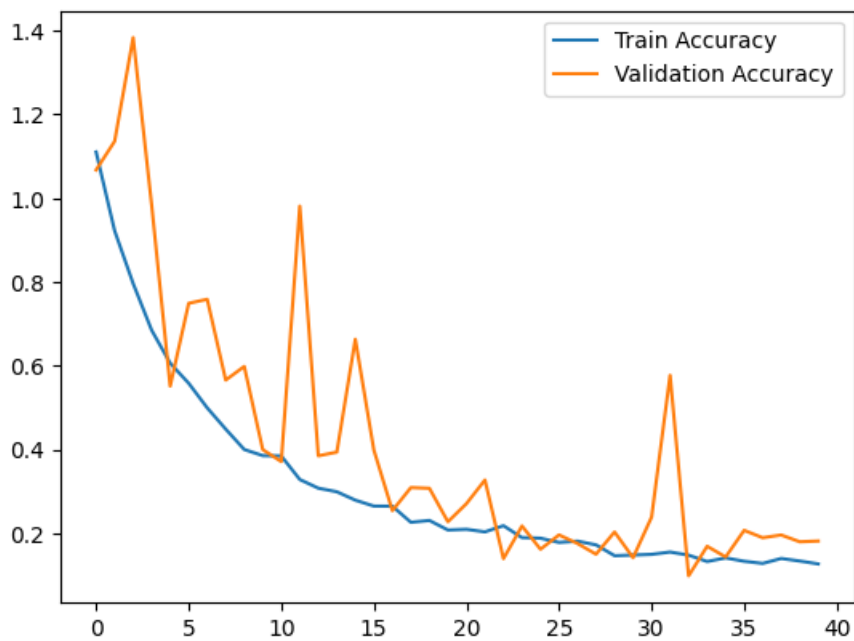
```
plt.legend()
```

```
plt.show()
```



#畫訓練損失跟驗證損失

```
plt.plot(history.history['loss'], label='Train Accuracy')
plt.plot(history.history['val_loss'], label='Validation Accuracy')
plt.legend()
plt.show()
```



#用訓練好的模型，對 test_data 裡的每一張圖做預測

```
y_prob=model.predict(test_data)
```

```

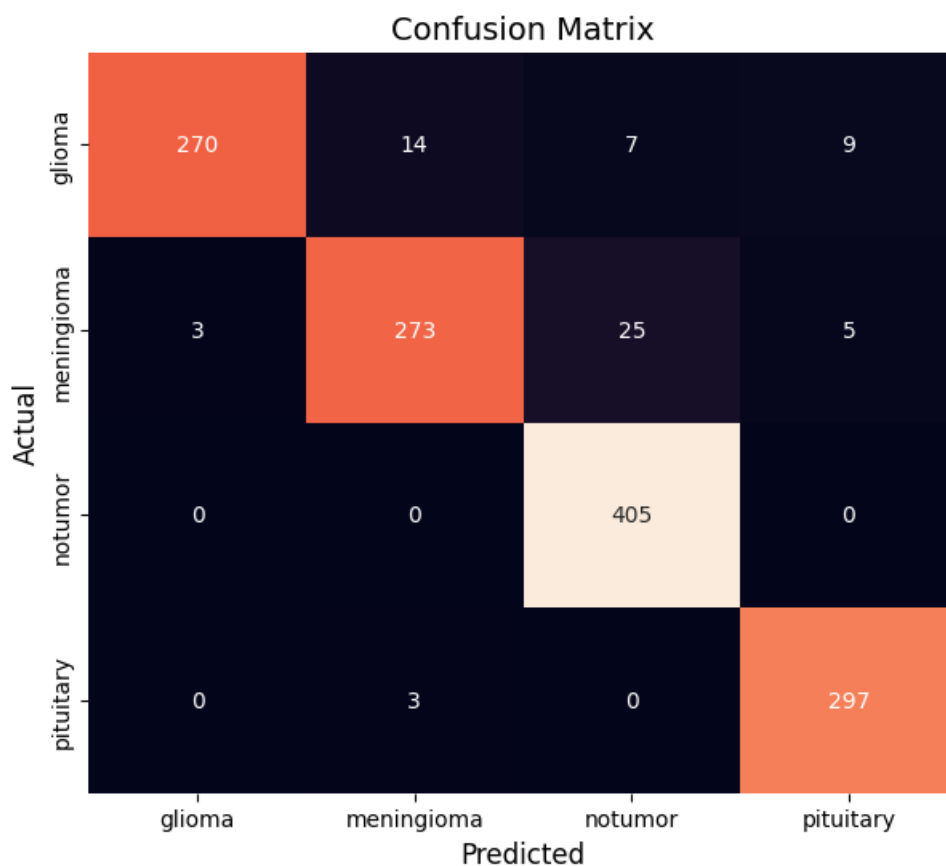
#準備真實標籤
y_true=test_data.classes

#把每一筆的機率分佈，轉成實際預測的類別編號
y_pred=np.argmax(y_prob,axis=1)

#計算並畫出分類模型的混淆矩陣
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_true,y_pred)
classes = ['glioma', 'meningioma', 'notumor', 'pituitary']

plt.figure(figsize=(7,6))
sns.heatmap(cm, annot=True, fmt='d', cbar=False,
            xticklabels=classes, yticklabels=classes)
plt.xlabel('Predicted', fontsize=12)
plt.ylabel('Actual', fontsize=12)
plt.title('Confusion Matrix', fontsize=14)
plt.show()

```



```

#前置：匯入與資料
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize
n_classes = 4
y_true_bin = label_binarize(y_true, classes=[0, 1, 2, 3])
#原始真實類別標籤
y_true = y_true
#模型對每個樣本、每個類別的預測機率
y_score = y_prob

#初始化字典，存每一類的 FPR/TPR/AUC
fpr = dict()
tpr = dict()
roc_auc = dict()

#逐類別計算 ROC 與 AUC
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

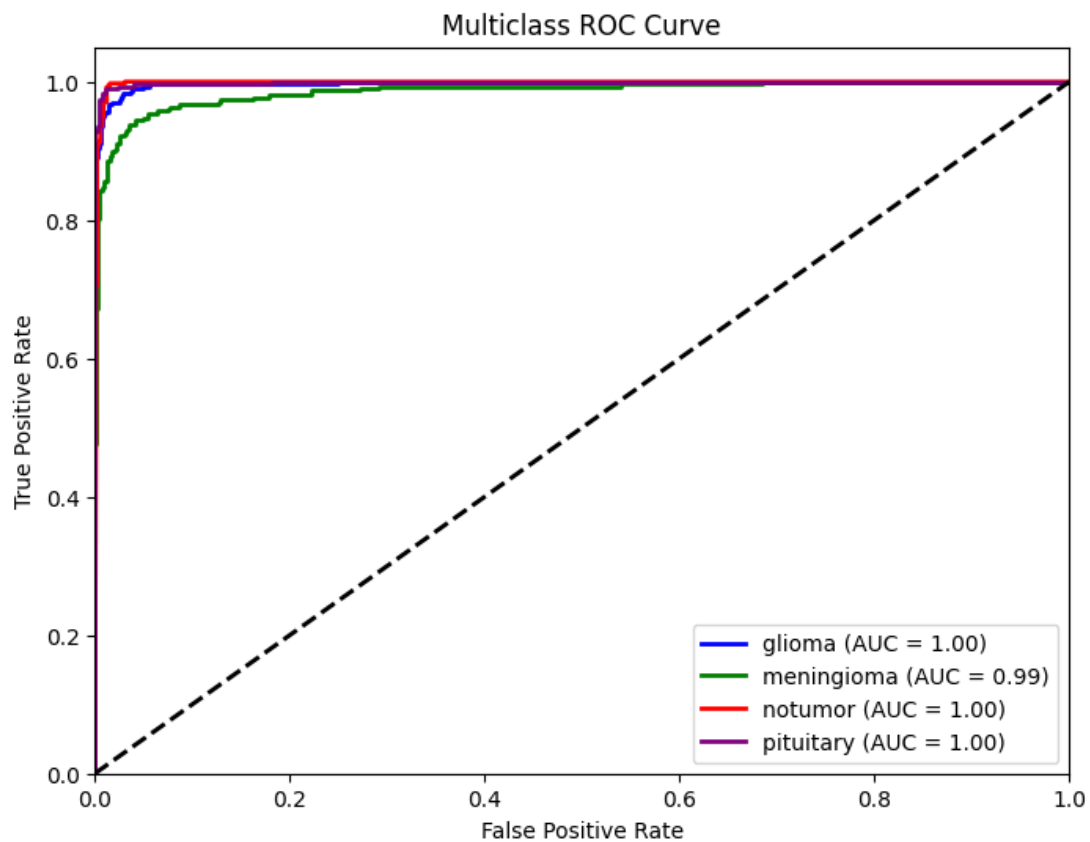
#準備畫圖
plt.figure(figsize=(8,6))
colors = ['blue', 'green', 'red', 'purple']
class_names = ['glioma', 'meningioma', 'notumor', 'pituitary']

#畫每一類的 ROC 曲線
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label=f'{class_names[i]} (AUC = {roc_auc[i]:0.2f})')

#畫對角線 & 美化座標軸
plt.plot([0, 1], [0, 1], 'k--', lw=2) # diagonal line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Multiclass ROC Curve')

```

```
plt.legend(loc='lower right')
plt.show()
```



```
from sklearn.metrics import f1_score, precision_score, recall_score
class_names = ['glioma', 'meningioma', 'notumor', 'pituitary']
```

```
#根據真實標籤 y_true 和模型預測 y_pred，計算分類表現的指標
```

```
#表示不要平均，而是回傳每一個類別各自的分數
```

```
precision = precision_score(y_true, y_pred, average=None)
```

```
recall = recall_score(y_true, y_pred, average=None)
```

```
f1 = f1_score(y_true, y_pred, average=None)
```

```
#為畫圖準備座標
```

```
x = np.arange(len(class_names))
```

```
width = 0.25
```

```
#畫出三種指標的長條圖
```

```
plt.figure(figsize=(9,6))
```



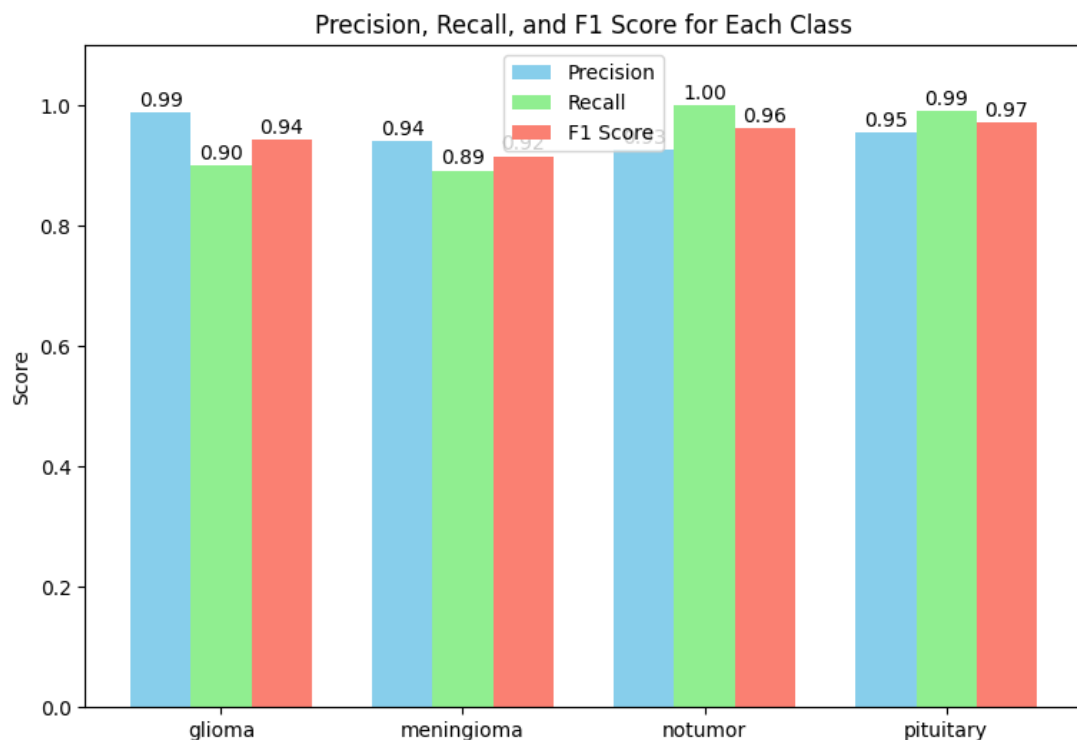
```

plt.bar(x - width, precision, width, label='Precision', color='skyblue')
plt.bar(x, recall, width, label='Recall', color='lightgreen')
plt.bar(x + width, f1, width, label='F1 Score', color='salmon')

#在長條上標數字
for i in range(len(class_names)):
    plt.text(x[i] - width, precision[i]+0.01, f'{precision[i]:.2f}', ha='center',
    fontsize=10)
    plt.text(x[i], recall[i]+0.01, f'{recall[i]:.2f}', ha='center', fontsize=10)
    plt.text(x[i] + width, f1[i]+0.01, f'{f1[i]:.2f}', ha='center', fontsize=10)

#設定座標軸與標題
plt.xticks(x, class_names)
plt.ylim(0, 1.1)
plt.ylabel('Score')
plt.title('Precision, Recall, and F1 Score for Each Class')
plt.legend()
plt.show()

```



#評估分類模型表現

```
from sklearn.metrics import classification_report
print(classification_report(y_true, y_pred,
target_names=list(test_data.class_indices.keys())))
```

	precision	recall	f1-score	support
glioma	0.99	0.90	0.94	300
meningioma	0.94	0.89	0.92	306
notumor	0.93	1.00	0.96	405
pituitary	0.95	0.99	0.97	300
accuracy			0.95	1311
macro avg	0.95	0.95	0.95	1311
weighted avg	0.95	0.95	0.95	1311

Q5. 自編碼器練習：請參照下面連結網頁的說明，試以

「**CelebFaces**」數據集建立自編碼器。

程式碼如下：

```
%matplotlib inline
%config InlineBackend.figure_format = 'retina'

#匯入資料庫
import time
import math
import random
import numpy as np
from scipy.stats import norm
import pylab as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.autograd import Variable
from torch.utils.data import Dataset, DataLoader
from torchvision import datasets, transforms
```

```

import os
import glob
from functools import partial
from IPython.display import display, clear_output
from ipywidgets import interact, interactive, fixed, interact_manual
import ipywidgets as widgets
from PIL import Image
from tqdm import tqdm

torch.set_default_dtype(torch.float32)
np.random.seed(42)
torch.manual_seed(42)
device = torch.device('cpu')

cuda = torch.cuda.is_available()
if cuda:
    torch.cuda.manual_seed(42)
    device = torch.device('cuda')

#影像的高度、寬度
input_shape = (64, 64)

#把訓練過程中跟 VAE / GAN 類型模型相關的各種東西畫在同一張圖上
class Visualizer(object):
    def __init__(self,
                 figsize,
                 loss_xlim=None,
                 loss_ylim=None,
                 plot_interval=1): #代表每隔多少個 epoch 才畫一次 loss 點
        self.loss_xlim = loss_xlim
        self.loss_ylim = loss_ylim
        self.plot_interval = plot_interval
        sns.set()
        self.fig = plt.figure(figsize=figsize)
        self.rows, self.cols = 4, 4

```

```

# 在一塊區域塞 4x4 圖片
def plot_subgrid(self, gs, images, title):
    outer_ax = plt.subplot(gs)
    outer_ax.set_title(title)
    outer_ax.set_xticks([])
    outer_ax.set_yticks([])

    inner_grid = gridspec.GridSpecFromSubplotSpec(
        self.rows, self.cols, subplot_spec=gs, wspace=.1, hspace=.1)

    for i, image in enumerate(images[:(self.rows * self.cols)]):
        ax = plt.Subplot(self.fig, inner_grid[i])
        ax.set_xticks([])
        ax.set_yticks([])
        #讓圖片比例為 1:1，不會被拉長或壓扁
        ax.set_aspect('equal')
        ax.imshow(image.cpu().permute(1, 2, 0).numpy())
        #把這個子圖真正加到 figure 裡
        self.fig.add_subplot(ax)

#一次畫出所有東西
def plot(self, x, x_recon, x_gen, z, loss_dict):

    gs = gridspec.GridSpec(2, 4, wspace=.2, hspace=.2,
                           height_ratios=(2, 1.5))

    #4x4 影像網格
    self.plot_subgrid(gs[0], x, 'data')
    self.plot_subgrid(gs[1], x_recon, 'reconstructed data')
    self.plot_subgrid(gs[2], x_gen, 'generated samples')

    ax_z = plt.subplot(gs[3])

    ax_z.scatter(z[:, 0], z[:, 1], 1)
    ax_z.set_title('latent samples')
    ax_z.set_aspect('equal', adjustable='datalim')
    #loss 走勢圖
    ax_trace = plt.subplot(gs[1, :])
    ax_trace.set_ylabel('loss')

```

```

ax_trace.set_xlabel('epochs')
if self.loss_xlim is not None:
    ax_trace.set_xlim(self.loss_xlim)
if self.loss_ylim is not None:
    ax_trace.set_ylim(self.loss_ylim)
for label, loss in loss_dict.items():
    plot_iter = np.arange(len(loss)) * self.plot_interval
    ax_trace.plot(plot_iter, loss, '-', label=label)
if len(loss_dict) > 1:
    ax_trace.legend(ncol=len(loss_dict), loc='upper center')
self.draw()

def draw(self):
    clear_output(wait=True)
    display(self.fig)

```

#把一張 PyTorch tensor 形式的圖片，轉成 NumPy 陣列，再用 Matplotlib 單獨顯示出來，並且關掉格線跟座標軸

```
def plot_image(image):
```

```

    plt.figure()
    plt.imshow(image.cpu().permute(1, 2, 0).numpy())
    plt.grid('off')
    plt.axis('off')
    plt.show()

```

#定義一個自訂的 CelebA 圖片資料集類別

```
class CelebA_50k(Dataset):
```

```

    def __init__(self, root, train=True, transform=None):
        self.train = train
        #用來指定要對每張圖片做的前處理
        self.transform = transform
        self.img_list = glob.glob(os.path.join(root, '*.jpg'))

```

#回傳這個資料集中有幾張圖片

```

def __len__(self):
    return len(self.img_list)

```

#讀出所有人臉圖片，轉成 tensor

```
def __getitem__(self, idx):  
    img_path = self.img_list[idx]  
    img = Image.open(img_path)  
    if self.transform is not None:  
        img = self.transform(img)  
    return img
```

#建立資料集物件與查看大小

```
data = CelebA_50k('/kaggle/input/50k-celeba-dataset-64x64/50k',  
transform=transforms.ToTensor())  
print(f"No. of images in (our) CelebA dataset: {len(data)}")
```

#隨機抽一張圖，印出 shape 再畫出來

```
test_image = data[random.randint(0, len(data))]  
print(f"Example Image with shape {test_image.shape}:")  
plot_image(test_image)
```

```
No. of images in (our) CelebA dataset: 50000  
Example Image with shape torch.Size([3, 64, 64]):
```



```

class Encoder(nn.Module):

    def __init__(self, input_features, latent_features):
        super(Encoder, self).__init__()

        # 輸入特徵維度
        self.input_features = input_features
        # 潛在向量的維度大小（輸出維度）
        self.latent_features = latent_features

        # 第一層卷積：輸入通道 3 (RGB)，輸出通道 32
        # 會把影像尺寸從 (3, 64, 64) 降成 (32, 32, 32)
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1)
        # ReLU 激活函數，增加非線性
        self.relu1 = nn.ReLU(inplace=True)

        # 第二層卷積：輸入通道 32，輸出通道 64
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1)
        # 第二個 ReLU
        self.relu2 = nn.ReLU(inplace=True)

        # 把卷積後的 feature map 攤平成一維向量
        # (batch_size, 64, 16, 16) → (batch_size, 64*16*16)
        self.flatten = nn.Flatten()

        # 全連接層：先把 64*16*16 維度壓成 256 維
        self.fc1 = nn.Linear(64*16*16, 256)
        self.relu3 = nn.ReLU(inplace=True)

        # 最後一層線性層：把 256 維壓成 latent_features 維

        self.fc2 = nn.Linear(256, latent_features)

    def forward(self, x):
        # 確保輸入 reshape 成 (batch_size, 3, 64, 64)
        x = x.view(-1, 3, 64, 64)

```

```
# 第一層卷積 + ReLU
x = self.conv1(x)
x = self.relu1(x)

# 第二層卷積 + ReLU
x = self.conv2(x)
x = self.relu2(x)

# 攤平成一維向量
x = self.flatten(x)

# 全連接層 + ReLU
x = self.fc1(x)
x = self.relu3(x)

# 映射到 latent_features 維的潛在空間
out = self.fc2(x)

# 回傳編碼後的向量
return out
```

```
#設定輸入輸出維度
input_features = 3*64*64
latent_features = 32
```

```
#建立編碼器 (Encoder)
encoder = Encoder(input_features, latent_features)
```

```
#建立一批隨機影像
x = torch.rand((100, 3, 64, 64))
```

```
## 把 100 張圖片輸入 encoder，得到壓縮後的 latent 向量 z
z = encoder(x)
#印出形狀
print(f"Input shape: {x.shape},\nZ shape: {z.shape}")
```

```
Input shape: torch.Size([100, 3, 64, 64]),
Z shape: torch.Size([100, 32])
```


#這段的功能是把隱含向量（latent vector）還原成原本的影像（RGB 64×64）

```
class Decoder(nn.Module):
```

```
    def __init__(self, output_features, latent_features):
        super(Decoder, self).__init__()
        self.output_features = output_features          #（預期）輸出維度
                                                         #（3*64*64），存起來但沒直接用
        self.latent_features = latent_features          # 編碼維度，decoder
                                                         端的輸入
```

```
    # 全連接層：把 latent 向量升成 256 維
```

```
    self.fc1 = nn.Linear(latent_features, 256)
```

```
    self.relu1 = nn.ReLU(inplace=True)
```

```
    # 全連接層：再升成和 Encoder flatten 一樣的空間（64*16*16）
```

```
    self.fc2 = nn.Linear(256, 64*16*16)
```

```
    self.relu2 = nn.ReLU(inplace=True)
```

```
    # Unflatten：把一維向量變回 shape = (64, 16, 16)
```

```
    self.unflatten = nn.Unflatten(1, (64, 16, 16))
```

```
    # 反卷積（上採樣）：把影像空間從 (64, 16, 16)升成 (32, 32, 32)
```

```
    self.conv1 = nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2,
padding=1, output_padding=1)
```

```
    self.relu3 = nn.ReLU(inplace=True)
```

```
    # 再一次反卷積：升成 (3, 64, 64)，回復成原始 RGB 圖片大小
```

```
    self.conv2 = nn.ConvTranspose2d(32, 3, kernel_size=3, stride=2, padding=1,
output_padding=1)
```

```
    def forward(self, z):
```

```
        # z 是 (batch_size, latent_dim) 向量
```

```
        z = self.fc1(z)          #→(batch_size, 256)
```

```
        z = self.relu1(z)
```

```
        z = self.fc2(z)          #→(batch_size, 64*16*16)
```

```
        z = self.relu2(z)
```

```
        z = self.unflatten(z)    #→(batch_size, 64, 16, 16)  # 恢復空間結構
```

```

        z = self.conv1(z)                #→(batch_size, 32, 32, 32)    # 第一次
upsampling
        z = self.relu3(z)
        z = self.conv2(z)                #→(batch_size, 3, 64, 64)    # 第二次
upsampling，還原原圖大小
        return z                        # 最終結果是重建回來的影像

```

#這段是用隨機的 latent 向量丟進 decoder，看看 decoder 輸出什麼形狀

```

output_features = 3*64*64
latent_features = 32
decoder = Decoder(output_features, latent_features)
z = torch.rand((100, 32))
x_recon = decoder(z)
print(f"Input shape: {z.shape} \nOutput shape: {x_recon.shape}")

Input shape: torch.Size([100, 32])
Output shape: torch.Size([100, 3, 64, 64])

```

```

class Autoencoder(nn.Module):

```

```

    #定義網路的結構

```

```

    def __init__(self, input_features, latent_features):
        super().__init__()

```

```

        self.input_features = self.output_features = input_features
        self.latent_features = latent_features

```

```

        self.encoder = Encoder(self.input_features, self.latent_features)
        self.decoder = Decoder(self.output_features, self.latent_features)
        self.relu = nn.ReLU()

```

```

        self.loss = nn.MSELoss()

```

```

    def forward(self, x):

```

```

        #編碼
        z = self.encoder(x)
        #激活函數處理
        x_recon = self.decoder(self.relu(z))
        #計算損失
        loss = self.loss(x_recon, x)
        #回傳結果
        return {
            'z': z.detach(),
            'x_recon': x_recon.detach(),
            'loss': loss,
        }

```

#決定維度是什麼

```
input_features = 3*64*64
```

```
latent_features = 32
```

#建立 Autoencoder 模型

```
model = Autoencoder(input_features, latent_features)
```

#計算可訓練參數的總數

```
num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
```

#輸出參數數量

```
print(f"No. of trainable parameters: {num_params}")
```

```
No. of trainable parameters: 8460643
```

#從資料集中隨機抽一張圖片、先畫出原圖，再丟進 autoencoder 做重建，最後把重建後的圖片畫出來

```
test_image = data[random.randint(0, len(data))]
```

```
print("Test image input:")
```

```
plot_image(test_image)
```

```
outputs = model(test_image.unsqueeze(0))
```

```
print("Test image reconstruction:")
```

```
plot_image(outputs['x_recon'].squeeze())
```

Test image input:



Test image reconstruction:



```
#模型與訓練參數
input_features = 3 * 64 * 64    #不用改
latent_features = 10
lr = 1e-3
batch_size = 128
num_epochs = 20

#初始化模型並搬到 GPU
model = Autoencoder(input_features, latent_features)
model = model.to(device)

#初始化資料集與 DataLoader
data = CelebA_50k('/kaggle/input/50k-celeba-dataset-64x64/50k',
transform=transforms.ToTensor())
train_loader = DataLoader(data, batch_size=batch_size, num_workers=4,
shuffle=True, pin_memory=True)
val_loader = DataLoader(data, batch_size=2000, num_workers=1, shuffle=False,
pin_memory=True)

#使用 Adam 這個 optimizer，負責根據梯度更新 model.parameters()（模型所有可學習參數）
optimizer = optim.Adam(model.parameters(), lr=lr)

#取出測試圖像並前向傳播（Forward）
print("BEFORE GRADIENT STEP:")
test_image = data[random.randint(0, len(data))]
test_image = test_image.unsqueeze(0).to(device)
output = model(test_image)

#清除舊的梯度
optimizer.zero_grad()

#計算損失並反向傳播（Backward）
loss_before = output['loss']
print("loss: ", loss_before.item())
```

```

loss_before.backward()
print()
print("AFTER GRADIENT STEP:")
#執行梯度下降法（更新權重）
optimizer.step()

#再做一次前向傳播，觀察 loss 是否減少
output = model(test_image)
loss_after = output['loss']
print("loss: ",loss_after.item())
print()
print("Difference in loss:", (loss_before - loss_after).item())
print("This should be some positive number to say we reduced loss")

```

```

BEFORE GRADIENT STEP:
loss:  0.18687115609645844

```

```

AFTER GRADIENT STEP:
loss:  0.18134908378124237

```

```

Difference in loss: 0.0055220723152160645

```

```

def train_autoencoder(model, train_loader, val_loader, optimizer, epochs=1,
seed=42):

```

```

    #設定隨機種子（random seed）
    torch.manual_seed(seed)
    if cuda:
        torch.cuda.manual_seed(seed)

```

```

    #初始化記錄和視覺化
    plot_interval = 1
    history = {'loss': [],}
    vis = Visualizer(
        figsize=(16, 7),
        loss_xlim=(0, epochs),
        plot_interval=plot_interval,

```

```

)

print("Started training... (first epoch can take a while)")
#訓練循環主體
for epoch in range(epochs):
    model = model.train()
    for batch_idx, batch_x in (enumerate(tqdm(train_loader, desc=f"Epoch
{epoch+1}/{epochs}"))):
        batch_x = batch_x.to(device)
        optimizer.zero_grad()
        pred = model(batch_x)
        loss = pred['loss']
        loss.backward()
        optimizer.step()

    #每隔一段時間驗證/視覺化
    if epoch % plot_interval == 0:
        test_autoencoder(model, val_loader, history, vis)

return model

#測試與可視化函式
@torch.no_grad()
def test_autoencoder(model, val_loader, history, vis):
    model = model.eval()
    batch_x = next(iter(val_loader)).to(device)
    loss_dict = model(batch_x)
    history['loss'].append(loss_dict['loss'].item())
    x_recon = loss_dict['x_recon']
    x_gen = model.decoder(torch.relu(torch.randn_like(loss_dict['z'])))
    vis.plot(batch_x, x_recon, x_gen, loss_dict['z'].cpu().numpy(), history)

#設定儲存權重的檔案路徑
weights_path = '/kaggle/working/celeba_ae10d.pt'
#記錄訓練時間
tic = time.time()

```

```
model = train_autoencoder(model, train_loader, val_loader, optimizer, num_epochs,
seed=3)
```

```
toc = time.time()
```

```
print(f"Finished Training. Time taken: {toc-tic:.2f}s")
```

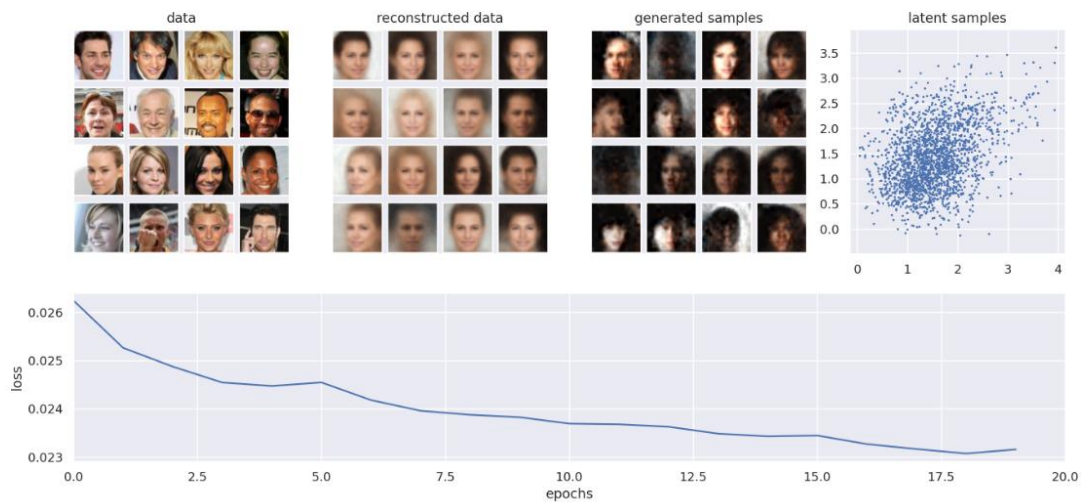
```
print(f"Saving weights to: {weights_path}")
```

```
#儲存模型權重
```

```
torch.save(model.state_dict(), weights_path)
```



```
Finished Training. Time taken: 3394.86s
Saving weights to: /kaggle/working/celeba_ae10d.pt
```



#這段程式主要是先用全連接層把低維的 z 展開成小的 feature map，再用一層一層的 ConvTranspose2d 做上採樣還原空間大小

```
class Decoder(nn.Module):
```



```

def __init__(self, output_features, latent_features):
    super(Decoder, self).__init__()
    self.output_features = output_features
    self.latent_features = latent_features

    self.fc1 = nn.Linear(latent_features, 256)
    self.fc2 = nn.Linear(256, 512)
    self.fc3 = nn.Linear(512, 1024)
    self.fc4 = nn.Linear(1024, 4096)

    self.conv1 = nn.ConvTranspose2d(256, 128, 4, 2, 1)
    self.conv2 = nn.ConvTranspose2d(128, 64, 4, 2, 1)
    self.conv3 = nn.ConvTranspose2d(64, 32, 4, 2, 1)
    self.conv4 = nn.ConvTranspose2d(32, 3, 4, 2, 1)

    self.relu = nn.ReLU()

```

```

def forward(self, z):

    z = self.relu(self.fc1(z))
    z = self.relu(self.fc2(z))
    z = self.relu(self.fc3(z))
    z = self.relu(self.fc4(z))
    z = z.view(-1, 256, 4, 4)

    z = self.relu(self.conv1(z))
    z = self.relu(self.conv2(z))
    z = self.relu(self.conv3(z))
    z = self.conv4(z)
    return z

```

#設定輸出/潛在特徵維度

output_features = 3*64*64

latent_features = 2

#產生一個解碼器物件，可以把 2 維 latent 向量還原成 64x64 RGB 圖片

decoder = Decoder(output_features, latent_features)

```
#建立 100 個 batch、每個是 2 維的隨機向量
z = torch.rand((100, 2))
#把這 100 筆 2 維向量用 Decoder 還原，得到 100 張圖片
x_logit = decoder(z)
print(f"Input shape: {z.shape} \nOutput shape: {x_logit.shape}")

Input shape: torch.Size([100, 2])
Output shape: torch.Size([100, 3, 64, 64])
```

```
class Encoder(nn.Module):
    #設定輸入特徵數和潛在空間維度
    def __init__(self, input_features, latent_features):
        super(Encoder, self).__init__()
        self.input_features = input_features
        self.latent_features = latent_features

        #建立四層卷積（每層都會讓特徵圖通道數增大、空間尺寸減小），用
        #來萃取高階影像特徵
        self.conv1 = nn.Conv2d(3, 32, 4, 2, 1)
        self.conv2 = nn.Conv2d(32, 64, 4, 2, 1)
        self.conv3 = nn.Conv2d(64, 128, 4, 2, 1)
        self.conv4 = nn.Conv2d(128, 256, 4, 2, 1)
        #展平並輸出分布參數
        self.fc_mu = nn.Linear(4096, latent_features)
        self.fc_logvar = nn.Linear(4096, latent_features)

    def forward(self, x):
        #前向傳播的卷積部分
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = F.relu(self.conv4(x))

        x = x.view(x.size(0), -1)
```

```

        mu = self.fc_mu(x)
        logvar = self.fc_logvar(x)
        #計算標準差，重參數化抽樣
        std = (logvar * .5).exp()
        z0 = self.rsample(mu, std)
        #回傳值
        return z0, mu, logvar
def rsample(self, mu, std):

```

```

    eps = torch.randn_like(mu)

```

```

    z = eps * std

```

```

    z = z + mu

```

```

    return z

```

#這兩個是模型超參數，用來決定 encoder 輸入向量長度

```
input_features = 3*64*64
```

```
latent_features = 2
```

#建立編碼器模組，屬於模型結構定義中的 encoder 部分

```
encoder = Encoder(input_features, latent_features)
```

#建立輸入 batch，模擬丟進 encoder 的影像資料

```
x = torch.rand((100, 3, 64, 64))
```

#呼叫 encoder 的 forward，mu 和 logvar 是 encoder 輸出的潛在分佈參數，z 是從中採樣出的 latent，全部都屬於 encoder 的輸出

```
z, mu, logvar = encoder(x)
```

```
print(f"Input shape: {x.shape},\nMean shape: {mu.shape}, Std shape: {logvar.shape},\nZ shape: {z.shape}")
```

```
Input shape: torch.Size([100, 3, 64, 64]),
```

```
Mean shape: torch.Size([100, 2]), Std shape: torch.Size([100, 2]), Z shape: torch.Size([100, 2])
```

```
class VAE(nn.Module):
```

```

    def __init__(self, input_features, latent_features):

```

```

super(VAE, self).__init__()
#輸入資料的維度
self.input_features = self.output_features = input_features
self.latent_features = latent_features
#Encoder 負責輸出 z, mu, logvar，Decoder 負責把 z 還原成輸入維度
的向量
self.encoder = Encoder(self.input_features, self.latent_features)
self.decoder = Decoder(self.output_features, self.latent_features)
#對輸入做 sigmoid，再算 BCE，這樣在數值上比較穩定，也不用自己
在 decoder 最後加 sigmoid 再算 BCELoss
self.recon_loss = nn.BCEWithLogitsLoss(reduction='sum')

self.kl_loss = lambda mu, logvar: -0.5 * (1 + logvar - mu ** 2 -
logvar.exp()).sum()

```

#用編碼器取得潛在分布參數與隨機潛在向量，解碼器重建輸入；計算總損失。輸出潛在特徵、重建結果和損失值

```

def forward(self, x):
    num_samples = x.size(0)
    z, mu, logvar = self.encoder(x)
    x_logit = self.decoder(z)
    recon_loss = self.recon_loss(x_logit, x) / num_samples
    kl_loss = self.kl_loss(mu, logvar) / num_samples
    loss = recon_loss + kl_loss
    return {
        'z': z.detach(),
        'x_recon': torch.sigmoid(x_logit.detach()),
        'loss': loss,
    }

```

#計算 VAE 模型的可訓練參數總數，也就是模型中可以透過梯度下降被優化的參數

```

input_features = 3*64*64
latent_features = 2
model = VAE(input_features, latent_features)
num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"No. of trainable parameters: {num_params}")

```

No. of trainable parameters: 6252487

#隨機挑一張圖片 → 顯示原圖 → 丟進模型做推論 → 顯示模型重建出的圖

```
test_image = data[random.randint(0, len(data))]
```

```
print("Test image input:")
```

```
plot_image(test_image)
```

```
outputs = model(test_image.unsqueeze(0))
```

```
print("Test image reconstruction:")
```

```
plot_image(outputs['x_recon'].squeeze())
```

Test image input:



Test image reconstruction:



#這段程式主要是設定並訓練一個用 CelebA 臉部資料集的變分自編碼器 (VAE) 模型時會用到的超參數、資料載入器與優化器

```
input_features = 3 * 64 * 64
```

```
latent_features = 2
```

```
lr = 1e-3
```

```
batch_size = 128
```

```
num_epochs = 50
```

```
model = VAE(input_features, latent_features)
```

```
model = model.to(device)
```

```
data = CelebA_50k('/kaggle/input/50k-celeba-dataset-64x64/50k',
```

```
transform=transforms.ToTensor())
```

```
train_loader = DataLoader(data, batch_size=batch_size, num_workers=4,
```

```
shuffle=True, pin_memory=True)
```

```
val_loader = DataLoader(data, batch_size=2000, num_workers=1, shuffle=False,
```

```
pin_memory=True)
```

```
optimizer = optim.Adam(model.parameters(), lr=lr)
```

#這段程式主要是檢查 VAE 訓練步驟前後損失(loss)的變化，透過單一訓練樣本進行梯度更新

```
print("BEFORE GRADIENT STEP:")
test_image = data[random.randint(0, len(data))]
test_image = test_image.unsqueeze(0).to(device)
output = model(test_image)
```

```
optimizer.zero_grad()
```

```
loss_before = output['loss']
print("loss: ", loss_before.item())
```

```
loss_before.backward()
```

```
print()
print("AFTER GRADIENT STEP:")
optimizer.step() # This takes the step to train
```

```
output = model(test_image)
```

```
loss_after = output['loss']
print("loss: ", loss_after.item())
```

```
print()
print("Difference in loss:", (loss_before - loss_after).item())
```

```
BEFORE GRADIENT STEP:
loss: 8577.5625
```

```
AFTER GRADIENT STEP:
loss: 8562.3896484375
```

```
Difference in loss: 15.1728515625
```

```

#訓練 VAE ( train_vae )
#train_vae 負責多輪 (epoch) 訓練與監控
def train_vae(model, train_loader, val_loader, optimizer, epochs=1, seed=42):

    torch.manual_seed(seed)
    if cuda:
        torch.cuda.manual_seed(seed)

    plot_interval = 1
    history = {'loss': []}
    vis = Visualizer(
        figsize=(16, 7),
        loss_xlim=(0, epochs),
        plot_interval=plot_interval,
    )

    print("Started training... (first epoch can take a while)")
    for epoch in range(epochs):
        model = model.train()
        for batch_idx, batch_x in enumerate(tqdm(train_loader, desc=f"Epoch
{epoch+1}/{epochs}")):
            batch_x = batch_x.to(device)
            optimizer.zero_grad()
            loss_dict = model(batch_x)
            loss_dict['loss'].backward()
            optimizer.step()

        if epoch % plot_interval == 0:
            test_vae(model, val_loader, history, vis)

    return model

#模型驗證/視覺化 ( test_vae )
# test_vae 負責驗證模型效果與可視化訓練進展
@torch.no_grad()
def test_vae(model, val_loader, history, vis):

```



```

model = model.eval()
batch_x = next(iter(val_loader)).to(device)

loss_dict = model(batch_x)
history['loss'].append(loss_dict['loss'].item())
x_recon = loss_dict['x_recon']
x_gen = torch.sigmoid(model.decoder(torch.randn_like(loss_dict['z'])))
vis.plot(batch_x, x_recon, x_gen, loss_dict['z'].cpu().numpy(), history)

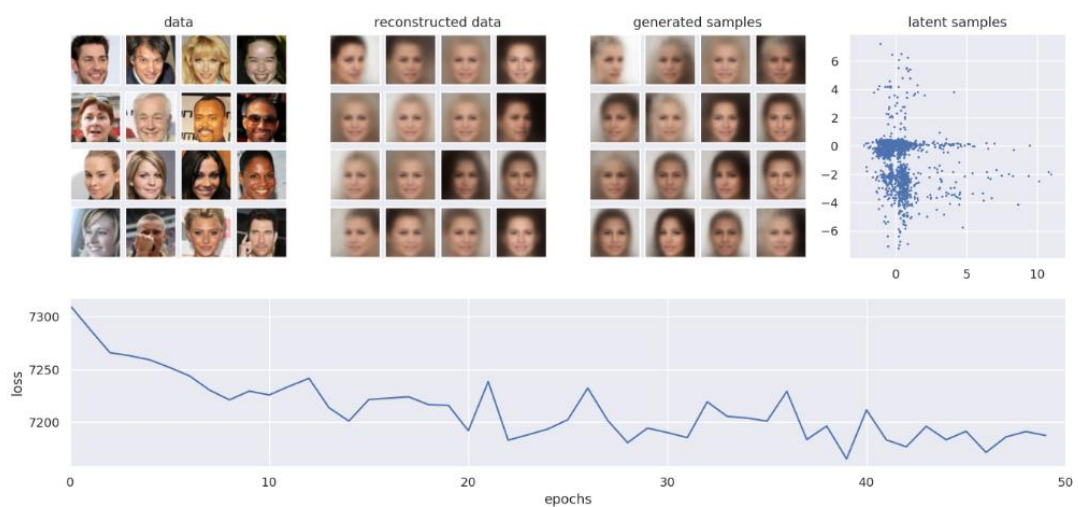
```

```

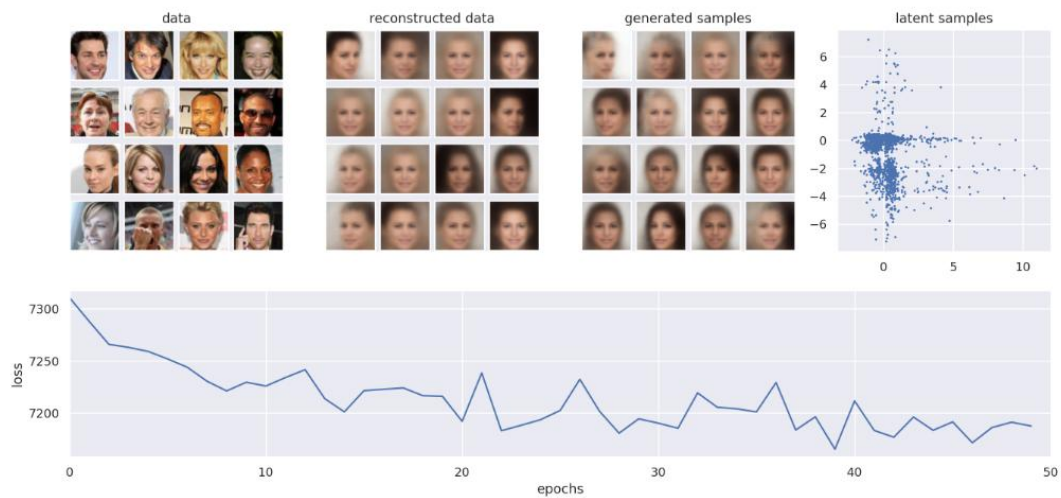
weights_path = '/kaggle/working/celeba_vae2d.pt'
tic = time.time()
#負責用訓練資料和驗證資料把模型訓練完
model = train_vae(model, train_loader, val_loader, optimizer, num_epochs, seed=3)
#用來算訓練總時間
toc = time.time()
print(f"Finished Training. Time taken: {toc-tic:.2f}s")

print(f"Saving weights to: {weights_path}")
#把目前模型的參數（weights、bias 等）存到你指定的 celeba_vae2d.pt 檔案裡
torch.save(model.state_dict(), weights_path)

```



```
Finished Training. Time taken: 1699.64s
Saving weights to: /kaggle/working/celeba_vae2d.pt
```



#設定 VAE 模型的超參數 → 建立並訓練模型 → 計算訓練時間 → 把訓練好的權重存到檔案

```
input_features = 3 * 64 * 64
```

```
latent_features = 15
```

```
lr = 1e-3
```

```
batch_size = 128
```

```
num_epochs = 20
```

```
model = VAE(input_features, latent_features)
```

```
model = model.to(device)
```

```
optimizer = optim.Adam(model.parameters(), lr=lr)
```

```
weights_path = '/kaggle/working/your_best_vae_model.pt'
```

```
tic = time.time()
```

```
model = train_vae(model, train_loader, val_loader, optimizer, num_epochs, seed=3)
```

```
toc = time.time()
```

```
print(f"Finished Training. Time taken: {toc-tic:.2f}s")
```

```
print(f"Saving weights to: {weights_path}")
```

```
torch.save(model.state_dict(), weights_path)
```



Finished Training. Time taken: 685.13s
 Saving weights to: /kaggle/working/your_best_vae_model.pt



#這段程式碼主要是如果有找到之前存好的 VAE 權重，就把它載入並切成推論模式；如果沒找到，就提示你沒檔案

```
weights_path = '/kaggle/working/your_best_vae_model.pt'
```

```
if(os.path.exists(weights_path)):
    model.load_state_dict(torch.load(weights_path))
    model = model.eval().to(device)
    print("Loaded pretrained VAE")
else:
    print(f"No weights found at: {weights_path}")
```

#這段程式碼主要是把模型切到推論模式，用訓練好的 **decoder**，選兩個潛在變數當滑桿，互動式掃描 **latent space** 並即時顯示對應生成圖片

```
dim1, dim2 = 2, 3
```

```
model = model.eval()
```

```
@torch.no_grad()
```

```
def generate(Z,decoder):
```

```
    Z = Z.to(device)
```

```
    x_gen = torch.sigmoid(decoder(Z)).cpu()
```

```
    return(x_gen)
```

```
def f(z1, z2):
```

```
    plt.grid(False)
```

```
    z = torch.zeros(latent_features)
```

```
    z[dim1], z[dim2] = z1, z2
```

```
    x = generate(z.unsqueeze(0), model.decoder).squeeze().permute(1, 2,  
0).numpy()
```

```
    plt.imshow(x, interpolation='nearest', vmin=0,vmax=1)
```

```
    plt.show()
```

```
interact(f,z1=(-4.0, 4.0), z2=(-4.0, 4.0), continuous_update=False);
```

